

LEAF: Compact and Efficient Blind Signature from Code-based Assumptions

Yi-Fu Lai¹ , Edoardo Persichetti² ,

¹ Ruhr-Universität Bochum

² Florida Atlantic University, USA

Abstract. Recently, Hanzlik, Lai, Paracucchi, Slamanig, Tang proposed several blind signature frameworks, collectively named Tanuki(s) (Asiacrypt’25), built upon cryptographic group actions. Their work introduces novel techniques and culminates in a concurrently secure blind signature framework. Straightforward instantiations based on CSIDH (CSI-FiSh) and LESS yield signature sizes of 4.5 KB and 64 KB respectively, providing the first efficient blind signatures in the isogeny-based and code-based literature allowing concurrent executions.

In this work, we improve the code-based instantiations by using the canonical form of linear equivalent codes by a careful treatment. However, the canonical form does not naturally support a group action structure, which is central to the security proofs of Tanuki(s). Consequently and unfortunately, the original security guarantees do not directly apply. To address this, we develop two distinct non-black-box reductions for both blindness and the one-more unforgeability. In the end, the improvements do not compromise the security.

This results in a concurrently secure code-based blind signature scheme with a compact signature size of 4.4 KB, which is approximately 1 % smaller than the isogeny-based one. We also provide a C implementation where the signing time is 99ms and 268 Mcycles on an Intel i7 2.3 GHz CPU. We also look forward to our approaches benefiting advanced constructions built on top of LESS in the future.

This is the preliminary version of the application.

1 Introduction

Blind signatures, first proposed by Chaum [28], are a fundamental cryptographic primitive that allows a signer to issue signatures on messages selected by the user, without learning the content being signed. This strong privacy property is central to numerous applications, ranging from classic settings such as e-cash, e-voting, and anonymous credentials, to modern systems including Privacy Pass [31], Tumblebit [40], Apple Private Relay [4], Apple Private Cloud Compute [5], and Microsoft U-Prove [50].

The defining privacy property of blind signatures is *blindness*: the signer cannot link the “interaction” with the user to the “resulting signature” or gain

any meaningful information about the signed message. In addition, blind signature schemes guarantee *unforgeability*, ensuring that users cannot generate valid signatures without the signer’s involvement. More precisely, this property is formalized as *one-more unforgeability*, which requires that no adversary can produce $\ell + 1$ valid signatures after interacting with the signing oracle at most ℓ times. The latter security notion can be further classified as either *concurrent* or *sequential*, depending on whether the adversary is allowed to initiate multiple signing sessions in parallel or only one at a time.

Practical blind signatures have been proposed from various classical assumptions, including RSA [28], discrete logarithms [52,38,26], or pairings [20,35,37]. Unfortunately, the post-quantum landscape remains less favorable when it comes to diversity of assumptions as well as efficiency.

Broadly speaking, there are two main approaches to constructing blind signatures. The first follows Fischlin’s generic template [34], which has been extensively explored in the post-quantum setting through a sequence of works [1,51,16,22]. Recent works [1,51] with 40-100 KB in size, culminating in that of Beullens et al. [16], have reduced the signature size to approximately 22 kilobytes in the lattice literature. These efficiency gains stem largely from advances in lattice-based NIZK proof systems for structured languages [21,49].

The main efficiency bottleneck for the approach lies in the use of generic zero-knowledge proofs (ZKPs) during interaction between signer and user. Signing times can range from nearly one hour [1] to about 20 seconds [16] for proving knowledge of the relevant hash relations. By using ZK-friendly hash functions, it is estimated that signing times could be brought below 1 second [16].

The other main approach is to adapt sigma-protocol-based signatures into blind signatures [39,43]. This line of work had long faced two key challenges: the scarcity of general transformation techniques and the lack of clear concurrent-security guarantees. For a considerable time, only two adapting methods were known—one for linear hash-function families [39] and one using a twist-type operation [43]. The more severe challenge lies in the absence of provable security in the concurrent setting, due to the exponential security loss inherent in the proofs. In fact, achieving provable concurrent security with perfect blindness in the classical counterpart required over two decades of research [12,42,53,27]. Recent two independent and parallel attacks [44] by Katsumata, Lai, Reichle, and [33] by Do, Hanzlik, Paracucchi have demonstrated concrete vulnerabilities in several promising post-quantum instantiations [2,3,43] under only four concurrent executions.

Fortunately, a recent breakthrough [36] by Hanzlik, Lai, Paracucchi, Slamanig, Tang et al. introduced several new frameworks, *Tanuki(s)*, for (non-commutative) group actions, enabling the first provable concurrently secure blind signatures for this paradigm in the post-quantum literature. Concrete instantiations using isogeny-based (CSI-FiSh [15]) and code-based (LESS [17]) group actions achieve signature sizes of 3.8 KB and 56 KB, respectively. Among the instantiations, while the code-based instantiation is over $100\times$ faster and does

not suffer from the scalability issues as the isogeny-based instantiations [45], its signature size is roughly $15\times$ larger, leading to an unsatisfactory situation.

Can we narrow this gap?

1.1 Technical Overview.

We use the canonical form for the linear equivalence problem to reduce signature size. The canonical form allows us to represent a monomial map μ between two codes in $\mathbb{F}_q^{k \times n}$ by the canonical forms of the codes together with two index sets $J, J' \subseteq [n]$ (see Proposition 3), representing coset classes. It thereby reduces the representation cost of μ from $n(\log_2(q-1) + \log_2(n))$ bits to $2 \cdot n$ bits.

While applying it to Framework 2 of Tanuki is straightforward, applying it to Framework 3 of Tanuki is tricky where we need two consecutive monomial mappings between three codes from the static public keys to the ephemeral public keys and to the commitment codes. To compress the monomial, it is necessary to represent the codes in the canonical form. However, F -equivalent codes do not remain F -equivalent for an arbitrary monomial mapping so that the second monomial will not map to the commitment code from the ephemeral code defined in the canonical form. Moreover, these modifications break the group action structure inherent to the linear equivalence problem, which is central to both the blindness and unforgeability proofs in [36].

To be more specific, in the linear equivalence problem, the monomial matrix group $\mathcal{M}_n$ acts on codes over $\mathbb{F}_q^{k \times n}$. In the blindness proof of [36], for any three codes within the same orbit, the number of monomial mappings between any two codes is the same, ensuring that the distribution of image codes remains uniform when monomials are sampled uniformly at random. In contrast, passing to canonical forms partitions the space into equivalence classes, where each class may contain a different number of codes. As a result, the output distribution becomes non-uniform, even when the underlying monomials are sampled uniformly. While using minimal entropy argument could achieve the statistical blindness, it is also not clear how to measure the minimal-entropy in this case.

Besides, the unforgeability proof [36] introduces and relies on the so-called “algebraic group action model.” Hence, regardless of the extent of the changes, the modified scheme will fall outside the scope of that proof framework.

To address these challenges, we adopt two distinct strategies. First, we rewrite the blindness proof by developing some properties of canonical forms. Rather than showing that, for a fixed signer’s view, the probability of generating a particular transcript is the same, we prove that for any two fixed signer views, the resulting signature distributions are identical. To further reduce the representation cost, we allow the user to adaptively adjust the randomness so that the row-reduced echelon forms of the codes are always systematic, ensuring that one of the index sets J' can always be fixed as $\{1, \dots, k\}$, which halves the cost of the representation as n bits. (An alternative approach here is to use the abort technique, which, however, would incur expensive overhead and tricky issues in

the proofs as pointed out in [39].) Similarly, to address the issue of the non- F -equivalent codes after a monomial map, we therefore enforce the user to update randomness adaptively while computing the canonical form. In both cases, we can also show that the adaptive modifications do not compromise blindness.

For one-more unforgeability, we provide non-black-box reductions from our modified schemes to the original **Tanuki** instantiations. Despite the absence of a strict group action structure, these reductions show that the hardness of forging signatures in our scheme is the same as that of the original instantiation.

1.2 Contribution

We present **LEAF(s)**³, code-based blind signature schemes with compact signature sizes, reducing those of the **Tanuki-LESS** instantiations by a factor of 14.5. This is marginally smaller (by about 1%) than the isogeny-based instantiation of **Tanuki** using **CSIDH-512**. This size gain comes with only a moderate computational cost: the signing and verification times increase by roughly 30% and 60-80% respectively compared to the original **LESS** instantiation of **Tanuki**. The main source of overhead is the additional computation of **CF**, the cost of which is 80% as an **RREF** computation on our platform. With the recent optimizations reported in [11] for the canonical form, this overhead is expected to drop to about 41% on specific platforms.

We provide a C implementation on an Intel Core i7-12700H @ 2.30 GHz, achieving signing in 268 million cycles (≈ 99 ms), resulting in competitive post-quantum blind signatures in the literature in the scope of the signature size and runtime.

While the public key sizes remain relatively large (0.7-0.9 MB), they are still fitting well for many blind-signature applications such as anonymous tokens and digital coins, where the key does not always need to be transmitted or stored together with each signature [50, 18, 4]. In such settings, the public key can be initialized once and deployed locally at the endpoint, stored in cache and updated occasionally or managed centrally by a certificate authority. In these cases, users only required to reference the key without specifying it.

Acknowledgments

Yi-Fu Lai is funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Germany's Excellence Strategy - EXC 2092 CASA - 390781972.

2 Preliminary

Notations. We let $\mathbb{N}$ denote the set of natural numbers. For $n \in \mathbb{N}$, we denote the set $\{1, \dots, n\}$ as $[n]$. For $\kappa \in \mathbb{N}$ we denote with S_κ the set of all the permutations of $[\kappa]$. If $\sigma, \sigma' \in S_\kappa$ are permutations then we denote with $\sigma \circ \sigma'$ their

³ **LEss** instantiation **Ameliorated** using canonical **F**orm

composition (as functions) from right to left. For a subset $J \subseteq [n]$, we denote the complement by $\bar{J} := [n] - J$.

For a generator matrix $\mathbf{G}$, we extend the matrix multiplication to an array $\mathbf{G}(\mathbf{M}_1, \dots, \mathbf{M}_n) := (\mathbf{G}\mathbf{M}_1, \dots, \mathbf{G}\mathbf{M}_n)$. Similarly, when the context is clear, we extend RREF in the same way.

2.1 LESS, Linear Equivalence and Canonical Form

We begin by recalling the basic concepts and key propositions from [30]. Let $(E, \circ)$ denote a group of *isometries*, i.e., maps on $\mathbb{F}_q^n$ that preserve the Hamming distance. Each $\psi \in E$ acts as an endomorphism on codewords: for any code $\mathcal{C}$, we write $\psi(\mathcal{C}) := \{\psi(\mathbf{c}) : \mathbf{c} \in \mathcal{C}\}$. This induces a natural equivalence relation on codes:

Definition 1. *Two linear codes $\mathcal{C}$ and $\mathcal{C}'$ are called E -equivalent if there exists $\psi \in E$ such that $\mathcal{C}' = \psi(\mathcal{C})$. We write $\mathcal{C} \sim_E \mathcal{C}'$.*

The choice of E determines the underlying computational assumption. In the LESS setting [17], E is taken to be the group of $n \times n$ monomial matrices, denoted $\mathcal{M}_n$. This leads to the core hardness assumption:

Definition 2 (Linear Equivalence Problem (LEP)). *Given two linear codes $\mathcal{C}, \mathcal{C}' \subseteq \mathbb{F}_q^n$ of length n and dimension k with $\mathcal{C} \sim_{\mathcal{M}_n} \mathcal{C}'$, find a monomial matrix $\mu \in \mathcal{M}_n$ such that $\mathcal{C}' = \mu(\mathcal{C})$.*

A central technical tool is the row-reduced echelon form (RREF), which provides a canonical matrix representation for each linear code. For a generator matrix $\mathbf{G}$ of a code $\mathcal{C}$, the matrix $\text{RREF}(\mathbf{G})$ uniquely determines $\mathcal{C}$ up to E -equivalence. We thereby extend the notation $\sim$ for the generator matrices. Throughout this work, we do not distinguish between a monomial group element and its corresponding matrix representation $\mathbf{Q} \in \mathcal{M}_n$; both notions will be used interchangeably as context is clear. Hence, the linear equivalence problem can be interpreted as follows: given two generator matrices $\mathbf{G}, \mathbf{G}'$ with $\mathbf{G} \sim_{\mathcal{M}_n} \mathbf{G}'$ (generate $\mathcal{M}_n$ -equivalent codes), the task is to find a monomial matrix $\mathbf{Q} \in \mathcal{M}_n$ such that $\mathbf{G}' = \text{RREF}(\mathbf{G}\mathbf{Q})$.

[30] introduces canonical form functions (CF) to enable signature size reduction. We briefly review the main properties below.

Proposition 1 ([30]). *Let $F = \mathcal{M}_k \oplus \mathcal{M}_{n-k} \leq \mathcal{M}_n$ be the subgroup of monomial matrices acting on the rows and columns of the non-systematic part of a generator matrix. Then there exists a canonical form function CF (implicitly parameterized by F) such that, on input a k -dimensional systematic code, outputs either another k -dimensional systematic code or $\perp$. Concretely, $\text{CF} : \mathbb{F}^{k \times (n-k)} \rightarrow \mathbb{F}^{k \times (n-k)} \cup \{\perp\}$, and the following hold:*

1. CF runs in time polynomial in n and $\log_2(q)$;
2. The codes generated by $(\mathbf{I}_k \mid \mathbf{A})$ and $(\mathbf{I}_k \mid \mathbf{A}')$ are F -equivalent if and only if there exist $\mathbf{M}_1 \in \mathcal{M}_k$ and $\mathbf{M}_2 \in \mathcal{M}_{n-k}$ such that $\mathbf{A} = \mathbf{M}_1 \mathbf{A}' \mathbf{M}_2$;

3. For any $\mathbf{A}$ with $\text{CF}(\mathbf{A}) \neq \perp$, the codes generated by $(\mathbf{I}_k \mid \text{CF}(\mathbf{A}))$ and $(\mathbf{I}_k \mid \mathbf{A})$ are F -equivalent;
4. For any two F -equivalent codes generated by $(\mathbf{I}_k \mid \mathbf{A})$ and $(\mathbf{I}_k \mid \mathbf{A}')$, we have $\text{CF}(\mathbf{A}) = \text{CF}(\mathbf{A}')$.

For convenience, we may abuse notation and write $\text{CF}(\mathbf{G}) := (\mathbf{I}_k \mid \text{CF}(\mathbf{A}))$ for a systematic generator matrix $\mathbf{G} = (\mathbf{I}_k \mid \mathbf{A})$ whenever $\text{CF}(\mathbf{A})$ is defined, and $\text{CF}(\mathbf{G}) := \perp$ otherwise. In fact, several choices for the subgroup F are possible [30]. Following [6], we set $F = \mathcal{M}_k \oplus \mathcal{M}_{n-k}$, which guarantees that the probability of CF outputting $\perp$ is negligible. With this choice, we obtain the following result.

Proposition 2 (Thm 8, [30]). *There exists an efficient compression function $\text{CosetRep} : \mathcal{M}_n \rightarrow S_{k,n}$ mapping any $\mu \in \mathcal{M}_n$ to the unique ordered size- k subset $J \subseteq \{1, \dots, n\}$ such that μ admits a decomposition $\mu = \phi \circ \chi$ with $\phi \in F$ and $\chi \in S_{k,n}$, where J records the positions mapped to the first k coordinates by χ .*

Moreover, CosetRep is constant on cosets: for any $\mu_0, \mu_1 \in \mathcal{M}_n$ with $\mu_0 F = \mu_1 F$, we have $\text{CosetRep}(\mu_0) = \text{CosetRep}(\mu_1)$.

Hence, one can represent the output of CosetRep as an n -bit string representing choosing an entry or not. For our case where $n = 2k$, the encoding is almost optimal compared to $\log_2 \binom{n}{k}$.

Proposition 3 (Prop 10 [30]). *If $(\mathcal{C}, \mathcal{C}')$ admits a solution for CF-LEP, then a solution for LEP on input $\mathcal{C}, \mathcal{C}'$ can be found in polynomial time.*

Concretely, say the two codes are generated by matrices $\mathbf{G}, \mathbf{G}' \in \mathbb{F}_q^{k \times n}$. If there exist size- k sets $J, J' \subseteq \{1, \dots, n\}$ such that

$$\text{CF}(\mathbf{G}_J^{-1} \cdot \mathbf{G}_{\bar{J}}) = \text{CF}((\mathbf{G}')_{J'}^{-1} \cdot (\mathbf{G}')_{\bar{J}'}),$$

then one can efficiently find a monomial matrix $\mu \in \mathcal{M}_n$ such that $\mathcal{C}' = \mu(\mathcal{C})$.

[36] introduces the “one-more” variant of the assumption.

Definition 3 (One-More-Vectorization Problem (OMVP)). *Let $\Pi = (\mathbf{G}, \mathcal{M}_n)$ be given. We define the one-more vectorization problem game OMVP with respect to a p.p.t. adversary $\mathcal{A}$ in Figure 1. We define the advantage of $\mathcal{A}$ in OMVP with respect to Π as $\text{Adv}_{\Pi, \mathcal{A}}^{\text{OMVP}}(\lambda) := \Pr[\text{OMVP}^{\mathcal{A}}(\lambda) = 1]$, where the probability is over the randomness of sampling over G in the game and the internal randomness of $\mathcal{A}$.*

2.2 Blind Signature Schemes

We recall the notion of blind signatures. We focus on a subclass called three-move blind signatures.

Definition 4. *A three-move blind signature scheme $\text{BS} = (\text{KeyGen}, \text{Signer}, \text{User}, \text{Verify})$ consists of the following p.p.t. algorithms:*

OMVP $_{\Pi}^A(\lambda)$	dLog($\mathbf{G}_y, \mathbf{G}_z$)	Chall()
$C = []$; $k = 0$;	$k = k + 1$;	$\mathbf{M} \leftarrow_{\$} \mathcal{M}_n$
$(\mathbf{G}, \mathcal{M}_n) \leftarrow \Pi$	if $\exists \mathbf{M} \in \mathcal{M}_n$ s.t.	$\mathbf{G}_y \leftarrow \text{RREF}(\mathbf{G}\mathbf{M})$;
$C' \leftarrow \mathcal{A}^{\text{Chall, dLog}}(\mathbf{G}, \mathcal{M}_n)$	$\text{RREF}(\mathbf{M}\mathbf{G}_y) = \text{RREF}(\mathbf{G}_z)$ then	$C = C \parallel [\mathbf{M}]$
return ($\text{RREF}(\mathbf{G}C)$	return g	return $\mathbf{G}_y$
$= \text{RREF}(\mathbf{G}C') \wedge k < C $)	else return $\perp$	

Fig. 1. The game OMVP and the oracles Chall, dLog.

Key generation. On input the security parameter 1^λ , the probabilistic algorithm KeyGen outputs a public key pk and a secret key sk .

Signer. The Signer = (Signer₁, Signer₂) consists of two algorithms:

Signer₁(sk): on input a secret key sk , the probabilistic algorithm Signer₁ outputs an internal state $\text{st}_{\text{Signer}}$ and a commitment R .

Signer₂($\text{sk}, c, \text{st}_{\text{Signer}}$): on input a challenge $c \in \mathcal{C}$, a secret key sk and an internal state $\text{st}_{\text{Signer}}$, the algorithm Signer₂ outputs a response s .

User. The User = (User₁, User₂) consists of two algorithms:

User₁(pk, m, R): on input a public key pk , a message m and a signer commitment R , the probabilistic algorithm User₁ outputs a challenge $c \in \mathcal{C}$ and an internal state st_{User} .

User₂($\text{pk}, s, \text{st}_{\text{User}}$): on input a public key pk , a signer response s and an internal state st_{User} , the probabilistic algorithm User₂ outputs a signature sig on message m .

Verification. On input a public key pk , a message m , and a signature sig , the verification algorithm Verify outputs 1 to indicate the prover is valid, and 0 otherwise.

We say that a three-move blind signature scheme BS is correct if for every λ , for every key pair $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(\lambda)$, for every message and signature pair (m, sig) generated from the interaction between an honest user and an honest signer, we always have $\text{Verify}(\text{pk}, m, \text{sig}) = 1$. We will omit “three-moves” and will say blind signature for brevity.

We consider blind signatures secure if they have the following two security properties. On the one hand, we want the signature output by the user to be not linkable to its identity: this is called the *blindness* property. On the other hand, after ℓ interactions with the signer, we require that the user cannot output more than ℓ valid signatures. This property is called *one-more-unforgeability*. We formalize it using the definition from [41].

Definition 5 (Malicious-Key Blindness). A blind signature scheme BS is blind under malicious key if there exists a negligible function $\text{negl}(\lambda)$ such that for all λ and all PPT adversaries $\mathcal{A}$, the following holds: $\text{Adv}_{\mathcal{A}}^{\text{blind}}(\lambda) \leq \text{negl}(\lambda)$,

where $\text{Adv}_{\mathcal{A}}^{\text{blind}}(\lambda) = \left| \Pr[\text{ExpBlind}(\lambda) = 1] - \frac{1}{2} \right|$ and the experiment ExpBlind is

ExpBlind (λ)	Init ($\tilde{m}_0, \tilde{m}_1, \tilde{pk}$)
$b \leftarrow \mathbb{S} \{0, 1\}$	$m_0 \leftarrow \tilde{m}_0, m_1 \leftarrow \tilde{m}_1$
$b_1 \leftarrow b; b_2 \leftarrow 1 - b$	$pk \leftarrow \tilde{pk}$
$b' \leftarrow \mathcal{A}^{\text{Init}, U_1, U_2}(\lambda)$	$\text{sess}_1 \leftarrow \text{sess}_2 \leftarrow \text{init}$
return $b = b'$	
U ₁ (sid, R)	U ₂ (sid, s)
if $sid \notin \{1, 2\} \vee \text{sess}_{sid} \neq \text{init}$: return $\perp$	if $\text{sess}_{sid} \neq \text{open}$: return $\perp$
$\text{sess}_{sid} \leftarrow \text{open}$	$s_{sid} \leftarrow s$
$R_{sid} \leftarrow R$	$\text{sig}_{b_{sid}} \leftarrow \text{User}_2(pk, R_{sid}, c_{sid}, s_{sid}, \text{st}_{sid})$
$(c_{sid}, \text{st}_{sid}) \leftarrow \text{User}_1(pk, R_{sid}, m_{b_{sid}})$	if $\text{sess}_1 = \text{sess}_2 = \text{closed}$:
return (sid, c_{sid})	if $\text{sig}_0 = \perp \vee \text{sig}_1 = \perp$: return ($\perp, \perp$)
	return ($\text{sig}_0, \text{sig}_1$)
	return (sid, closed)

Fig. 2. Malicious-Key Blindness experiment **ExpBlind**.

defined in Figure 2. When $\text{Adv}_{\mathcal{A}}^{\text{blind}}(\lambda) = 0$, we say the blind signature scheme BS has perfect blindness.

Definition 6 (Concurrent ℓ -one-more unforgeability). A blind signature BS is ℓ -one-more unforgeable in the concurrent setting if there exists a negligible function $\text{negl}(\lambda)$ such that for all λ and all PPT adversaries $\mathcal{A}$, the following holds: $\text{Adv}_{\mathcal{A}, \ell}^{\text{cOMUF}}(\lambda) \leq \text{negl}(\lambda)$, where $\text{Adv}_{\mathcal{A}, \ell}^{\text{cOMUF}}(\lambda) = \Pr[\text{cOMUF}(\lambda) = 1]$ and cOMUF is the experiment defined in Figure 3.

Definition 7 (Sequential ℓ -one-more unforgeability). A blind signature BS is ℓ -one-more unforgeable in the sequential setting if there exists a negligible function $\text{negl}(\lambda)$ such that for all λ and all PPT adversaries $\mathcal{A}$, the following holds: $\text{Adv}_{\mathcal{A}, \ell}^{\text{sOMUF}}(\lambda) \leq \text{negl}(\lambda)$, where $\text{Adv}_{\mathcal{A}, \ell}^{\text{sOMUF}}(\lambda) = \Pr[\text{sOMUF}(\lambda) = 1]$, where sOMUF is similar to the experiment cOMUF defined in Figure 3 except with an additional restriction. The adversary is not allowed to query the Sign_1 oracle if there already exists an open session, i.e., the oracle returns $\perp$ if there exists a $k \leq \text{sid}$ for which $\text{sess}_k = \text{open}$.

3 Tanuki Frameworks and the Instantiation using Classic LESS

3.1 Tanuki-2-LESS: Sequential Secure Blind Signature

This section introduces the code-based instantiations of Tanuki-2 and Tanuki-3 given in [36] and the unforgeability properties. We leave the framework in the language of group actions in Appendix A.

cOMUF(λ)	Sign ₁ ($\cdot$)
$(sk, pk) \leftarrow \text{KeyGen}(\lambda)$	$sid \leftarrow sid + 1$
$sid \leftarrow 0$	$sess_{sid} \leftarrow \text{open}$
$((m_1, sig_1), \dots, (m_\ell, sig_\ell)) \leftarrow \mathcal{A}^{\text{Sign}_1, \text{Sign}_2}(pk)$	$(st_{sid}, R_{sid}) \leftarrow \text{Signer}_1(sk)$
if $\exists i \neq j : m_i = m_j$: return 0	return (sid, R_{sid})
if $\exists i \in [\ell] : \text{Verify}(pk, m, sig_i) = 0$: return 0	
$Q_{\text{Sign}_2} \leftarrow \{k sess_k = \text{closed}\} $	Sign ₂ (sid, c)
if $\ell \geq Q_{\text{Sign}_2} + 1$: return 1	if $sess_{sid} \neq \text{open}$: return $\perp$
return 0	$sess_{sid} = \text{closed}$
	$s_{sid} \leftarrow \text{Signer}_2(sk, st_{sid}, R_{sid}, c)$
	return s_{sid}

Fig. 3. Unforgeability experiment cOMUF.

Theorem 1. *Scheme 1, the instantiation of Framework 1, is sequentially OMUF under the random oracle model and the algebraic group action model (see Appendix A.2) under the one-more vectorization problem.*

Let q be a prime number, $G = \mathcal{M}_n$, and γ, κ polynomial in λ . Let $H : \{0, 1\}^* \rightarrow [\kappa]^\gamma$ be a hash function such that for any string m and any $i, j \in [\kappa]$, the number of occurrences of i and j in $H(m)$ are equal.

KeyGen(λ): Generate a random full-rank matrix $\mathbf{G} \in \mathbb{F}_q^{k \times n}$. Set $\mathbf{Q}_1 = \text{id}$ and $\text{pk}_1 = \text{RREF}(\mathbf{G})$. Independently sample $\mathbf{Q}_2, \dots, \mathbf{Q}_\kappa \leftarrow \$ G$, and set $\text{pk}_i = \text{RREF}(\mathbf{G}\mathbf{Q}_i)$ for $i = 2, \dots, \kappa$. Output $\text{sk} = (\mathbf{Q}_2, \dots, \mathbf{Q}_\kappa)$ and $\text{pk} = (\text{pk}_1, \dots, \text{pk}_\kappa)$.

Signer₁(sk): Sample $\tilde{\mathbf{Q}}_1, \dots, \tilde{\mathbf{Q}}_\gamma \leftarrow \$ G$. For each $i \in [\gamma]$, compute $\mathbf{A}_i = \text{RREF}(\mathbf{G}\tilde{\mathbf{Q}}_i)$. Send $Y = (\mathbf{A}_1, \dots, \mathbf{A}_\gamma)$ to the user and keep $(\tilde{\mathbf{Q}}_1, \dots, \tilde{\mathbf{Q}}_\gamma)$ privately.

User₁(pk, Y, m): For each $i \in [\gamma]$, sample $\mathbf{R}_i \leftarrow \$ G$ and compute $\mathbf{B}_i = \text{RREF}(\mathbf{A}_i\mathbf{R}_i)$. Choose a random permutation $\pi : [\gamma] \rightarrow [\gamma]$. Compute the challenge $C = H(\mathbf{B}_{\pi(1)}, \dots, \mathbf{B}_{\pi(\gamma)}, m)$, interpret C as $(c_1, \dots, c_\gamma)$, and define $C' = (c_{\pi^{-1}(1)}, \dots, c_{\pi^{-1}(\gamma)})$. Send C' to the signer.

Signer₂(sk, C', ($\tilde{\mathbf{Q}}_1, \dots, \tilde{\mathbf{Q}}_\gamma$)): For each $i \in [\gamma]$, compute $\mathbf{Q}'_i = \mathbf{Q}_{c'_i}^{-1}\tilde{\mathbf{Q}}_i$. Send $S' = (\mathbf{Q}'_1, \dots, \mathbf{Q}'_\gamma)$ to the user.

User₂(pk, S', π , ($\mathbf{R}_1, \mathbf{B}_1, \dots, \mathbf{R}_\gamma, \mathbf{B}_\gamma$), m): For each $i \in [\gamma]$, compute $\hat{\mathbf{Q}}_i = \mathbf{Q}'_i\mathbf{R}_i$ and $\mathbf{A}_i^* = \text{RREF}(\text{pk}_{c'_i}\hat{\mathbf{Q}}_i)$. If $\mathbf{A}_i^* \neq \mathbf{B}_i$ for any i , abort. Output signature $\text{sig} = (C, (\hat{\mathbf{Q}}_{\pi(1)}, \dots, \hat{\mathbf{Q}}_{\pi(\gamma)}))$.

Verify(pk, m, sig): Parse $\text{sig} = ((c_1, \dots, c_\gamma), (\hat{\mathbf{Q}}_1, \dots, \hat{\mathbf{Q}}_\gamma))$. For each $i \in [\gamma]$, compute $\mathbf{B}_i = \text{RREF}(\text{pk}_{c_i}\hat{\mathbf{Q}}_i)$. Verify if $C = H(\mathbf{B}_1, \dots, \mathbf{B}_\gamma, m)$ holds. Output 1 if the check passes, 0 otherwise.

Scheme 1: Instantiation of Tanuki-2 using Classic LESS

3.2 Tanuki-3-LESS: Concurrent Secure Blind Signature

Theorem 2. *Scheme 2, the instantiation of Framework 2, is sequentially OMUF under the random oracle model and the algebraic group action model (see Appendix A.2) under the one-more vectorization problem.*

Let q be a prime, $G = \mathcal{M}_n$ denote the group of $n \times n$ monomial matrices over $\mathbb{F}_q$. Set polynomially-bounded parameters $\kappa \geq 2$, $\alpha > 0$, and $\gamma = \alpha\kappa$ as in the framework. Let $H : \{0, 1\}^* \rightarrow [\kappa]^\gamma$ be a hash function with balanced output, i.e., each symbol in $[\kappa]$ appears equally often.

KeyGen(λ): Sample a random full-rank matrix $\mathbf{G} \in \mathbb{F}_q^{k \times n}$. Let $\mathbf{Q}_1 = \text{id}$ and set $\text{pk}_1 = \text{RREF}(\mathbf{G})$. For $i = 2$ to κ , sample $\mathbf{Q}_i \leftarrow \$ G$ and set $\text{pk}_i = \text{RREF}(\mathbf{G}\mathbf{Q}_i)$. Output $\text{sk} = (\mathbf{Q}_2, \dots, \mathbf{Q}_\kappa)$ and $\text{pk} = (\text{pk}_1, \dots, \text{pk}_\kappa)$.

KeyRand(pk): Given $\text{pk} = (\text{pk}_1, \dots, \text{pk}_\kappa)$, sample $\mathbf{T}_1, \dots, \mathbf{T}_\kappa \leftarrow \$ G$ and a permutation $\sigma \leftarrow \$ S_\kappa$. For each $i \in [\kappa]$, set $\text{pk}'_i = \text{RREF}(\text{pk}_{\sigma(i)} \mathbf{T}_{\sigma(i)})$. Output $(\text{pk}', (\mathbf{T}_1, \dots, \mathbf{T}_\kappa), \sigma)$ where $\text{pk}' = (\text{pk}'_1, \dots, \text{pk}'_\kappa)$.

Signer₁(sk, pk): Run $(\text{pk}', T', \sigma') \leftarrow \text{KeyRand}(\text{pk})$ with $T' = (\mathbf{T}'_1, \dots, \mathbf{T}'_\kappa)$. For each $i \in [\kappa]$, set $\mathbf{Q}'_i = \mathbf{Q}_{\sigma'(i)} \mathbf{T}'_{\sigma'(i)}$. Sample $\tilde{\mathbf{Q}}_1, \dots, \tilde{\mathbf{Q}}_\gamma \leftarrow \$ G$ and compute $\mathbf{A}_i = \text{RREF}(\mathbf{G}\tilde{\mathbf{Q}}_i)$ for $i \in [\gamma]$. Send $(\text{pk}', (\mathbf{A}_1, \dots, \mathbf{A}_\gamma))$ to the user and keep $\tilde{\mathbf{Q}}_1, \dots, \tilde{\mathbf{Q}}_\gamma, \mathbf{Q}'_1, \dots, \mathbf{Q}'_\kappa, T', \sigma'$ as secret state.

User₁($\text{pk}', (\mathbf{A}_1, \dots, \mathbf{A}_\gamma), m$): Run $(\text{pk}'', T, \sigma) \leftarrow \text{KeyRand}(\text{pk}')$ with $T = (\mathbf{T}_1, \dots, \mathbf{T}_\kappa)$. Sample $\mathbf{R}_1, \dots, \mathbf{R}_\gamma \leftarrow \$ G$ and a random permutation π . For each $i \in [\gamma]$, compute $\mathbf{B}_i = \text{RREF}(\mathbf{A}_i \mathbf{R}_i)$. Let $c = H(\mathbf{B}_{\pi(1)}, \dots, \mathbf{B}_{\pi(\gamma)}, \text{pk}'', m)$ and parse $c = (c_1, \dots, c_\gamma)$. For each $i \in [\gamma]$, set $c'_i = \sigma(c_{\pi^{-1}(i)})$. Send $c' = (c'_1, \dots, c'_\gamma)$ to the signer.

Signer₂($c', \tilde{\mathbf{Q}}_1, \dots, \tilde{\mathbf{Q}}_\gamma, T', \sigma'$): For each $i \in [\gamma]$, compute $\mathbf{S}'_i = (\mathbf{Q}'_{c'_i})^{-1} \tilde{\mathbf{Q}}_i$. Return $(S' = (\mathbf{S}'_1, \dots, \mathbf{S}'_\gamma), T', \sigma')$ to the user.

User₂($\text{pk}, S', \pi, (\mathbf{R}_1, \dots, \mathbf{R}_\gamma), T, \sigma, T', \sigma', m$): Verify if $\text{KeyRand}(\text{pk}; T', \sigma') \stackrel{?}{=} \text{pk}'$; abort if not. For each $i \in [\gamma]$, verify that $\text{RREF}(\text{pk}'_{c'_i} \cdot \mathbf{S}'_i) \stackrel{?}{=} \mathbf{A}_i$; abort if not. For each $i \in [\gamma]$, set $\mathbf{S}_i = \mathbf{T}_{c'_i}^{-1} \mathbf{S}'_i \mathbf{R}_i$. Set $\tau = \sigma' \circ \sigma$. Output the signature

$$\text{sig} = (c, (\mathbf{S}_{\pi(1)}, \dots, \mathbf{S}_{\pi(\gamma)}), (\mathbf{T}'_{\sigma'(\sigma(1))} \mathbf{T}_{\sigma(1)}, \dots, \mathbf{T}'_{\sigma'(\sigma(\kappa))} \mathbf{T}_{\sigma(\kappa)}), \tau).$$

Verify(pk, m, sig): Parse $\text{sig} = (c = (c_1, \dots, c_\gamma), (\mathbf{S}_1, \dots, \mathbf{S}_\gamma), (\mathbf{T}_1^*, \dots, \mathbf{T}_\kappa^*), \tau)$. For each $i \in [\kappa]$, compute $\overline{\text{pk}}_i = \text{RREF}(\text{pk}_{\tau(i)} \mathbf{T}_i^*)$. For each $i \in [\gamma]$, compute $\mathbf{B}_i = \text{RREF}(\overline{\text{pk}}_{c_i} \mathbf{S}_i)$. Accept if and only if $c \stackrel{?}{=} H(\mathbf{B}_1, \dots, \mathbf{B}_\gamma, \overline{\text{pk}}, m)$ holds.

Scheme 2: Instantiation of Tanuki-3 (Randomized Multi-key) using Classic LESS

4 Amelioration using Canonical Form

We present improvements to the schemes based on canonical forms and the adaptive subroutines. Though the improvement for Scheme 2 will imply the corresponding improvement for Scheme 1, we present them separately to maintain clarity and pedagogical accessibility. Readers already familiar with the literature may skip Section 4.2.

4.1 Useful Propositions

Next, we introduce a few useful propositions supporting the security of our schemes. We first show a useful lemma which is implicitly used in [30].

Lemma 1. *Let $\mathcal{C}_0, \mathcal{C}_1$ be two linear codes where $\mathcal{C}_0$ is generated by $\mathbf{G}_0$ and $\mathcal{C}_1$ is generated by a systematic matrix $\mathbf{G}_1 = (\mathbf{I}_k \mid \mathbf{A}_1)$. Suppose $\mu \in \mathcal{M}_n$ satisfies $\mu(\mathcal{C}_0) = \mathcal{C}_1$, and let $J \leftarrow \text{CosetRep}(\mu)$. Then we have*

$$\text{CF}\left(\left((\mathbf{G}_0)_J\right)^{-1} \cdot (\mathbf{G}_0)_{\bar{J}}\right) = \text{CF}(\mathbf{A}_1).$$

Proof. Since $\mu(\mathcal{C}_0) = \mathcal{C}_1$, there exists $\mathbf{S} \in \text{GL}_k(q)$ and a monomial matrix $\mathbf{Q} \in \mathcal{M}_n$ such that

$$\mathbf{G}_1 = \mathbf{S} \cdot \mathbf{G}_0 \cdot \mathbf{Q}.$$

Recall that $J \leftarrow \text{CosetRep}(\mu)$ represents the size- k subset corresponding to the information set mapped to the first k coordinates by $\mathbf{Q}$. Hence, $(\mathbf{G}_0)_J$ is invertible. By following the decomposition through J and the standard linear algebra, we have

$$\mathbf{G}_1 = (\mathbf{I}_k \mid \mathbf{A}_1) = \mathbf{S} \cdot ((\mathbf{G}_0)_J \mid (\mathbf{G}_0)_{\bar{J}}) \cdot \begin{pmatrix} \mathbf{Q}'_r & \mathbf{0} \\ \mathbf{0} & \mathbf{Q}'_c \end{pmatrix}.$$

Thus, the non-systematic part of $\mathbf{G}_1$ can be expressed as

$$\mathbf{A}_1 = \mathbf{Q}'_r{}^{-1} \cdot (\mathbf{G}_0)_J^{-1} (\mathbf{G}_0)_{\bar{J}} \cdot \mathbf{Q}'_c,$$

where $\mathbf{Q}'_r{}^{-1} \in \mathcal{M}_k$ and $\mathbf{Q}'_c \in \mathcal{M}_{n-k}$. By Proposition 1, we have

$$\text{CF}\left((\mathbf{G}_0)_J^{-1} (\mathbf{G}_0)_{\bar{J}}\right) = \text{CF}(\mathbf{A}_1).$$

□

We further introduce RREF^* , an extension of the standard RREF . Given a generator matrix $\mathbf{G}$, the deterministic algorithm RREF^* outputs a systematic form $\mathbf{G}' \in \mathbb{F}_q^{k \times n}$ together with a permutation ρ satisfying $\mathbf{G}' = \text{RREF}(\mathbf{G}\rho)$. As will be seen later, this enables us to compress the representation cost without resorting to the abort technique, thereby avoiding the drawbacks highlighted in [39].

We introduce the following lemma essential for the blindness proofs.

Theorem 3. For any three $\mathcal{M}_n$ -equivalent codes generated by $\mathbf{G}$, $\mathbf{G}_0 = (\mathbf{I}_k \mid \mathbf{A}_0)$ and $\mathbf{G}_1 = (\mathbf{I}_k \mid \mathbf{A}_1)$ where $\text{CF}(\mathbf{G}_0) = \mathbf{G}_0$ and $\text{CF}(\mathbf{G}_1) = \mathbf{G}_1$, the following two sets have the same cardinality:

$$S_0 = \{\mathbf{Q} \in \mathcal{M}_n : \text{CF}(\mathbf{G}') = \mathbf{G}_0 \text{ where } (\mathbf{G}', \rho) \leftarrow \text{RREF}^*(\mathbf{G}\mathbf{Q})\}$$

and

$$S_1 = \{\mathbf{Q} \in \mathcal{M}_n : \text{CF}(\mathbf{G}') = \mathbf{G}_1 \text{ where } (\mathbf{G}', \rho) \leftarrow \text{RREF}^*(\mathbf{G}\mathbf{Q})\}$$

Proof. Due to the $\mathcal{M}_n$ -equivalence, there exists $\mathbf{M} \in \mathcal{M}_n$ such that $\mathbf{G}_1 = \text{RREF}(\mathbf{G}_0\mathbf{M})$. For each $\mathbf{Q} \in S_1$ together with the corresponding $(\mathbf{G}', \rho) \leftarrow \text{RREF}^*(\mathbf{G}\mathbf{Q})$, we claim that the mapping $\mathbf{Q} \mapsto \mathbf{Q}\rho\mathbf{M}\rho^{-1}$ is a bijection onto S_0 . First, the monomial $\mathbf{Q}\rho\mathbf{M}\rho^{-1}$ is in S_0 because

$$\text{RREF}(\mathbf{G}(\mathbf{Q}\rho\mathbf{M}\rho^{-1})\rho, \rho) \leftarrow \text{RREF}^*(\mathbf{Q}\rho\mathbf{M}\rho^{-1}).$$

Since $\text{RREF}(\mathbf{G}\mathbf{Q}\rho) = \mathbf{G}'$, and $\mathbf{G}\mathbf{Q}\rho$ is F -equivalent to $\mathbf{G}_0$. Hence,

$$\mathbf{G}\mathbf{Q}\rho\mathbf{M} = \mathbf{G}(\mathbf{Q}\rho\mathbf{M}\rho^{-1})\rho$$

is F -equivalent to $\mathbf{G}_0\mathbf{M}$.

Given $\mathbf{Q}_1 \in S_1$ with ρ be the corresponding auxiliary outputs via $\text{RREF}^*(\mathbf{G}\mathbf{Q}_1)$, take $\mathbf{Q}_0 = \mathbf{Q}_1\rho\mathbf{M}^{-1}\rho^{-1}$. Since $\text{RREF}(\mathbf{G}_1\mathbf{M}^{-1})$ is systematic, ρ is the auxiliary outputs of $\text{RREF}^*(\mathbf{G}\mathbf{Q}_0)$ such that $\text{RREF}(\mathbf{G}\mathbf{Q}_1\rho\mathbf{M}^{-1})$ is systematic. Hence, $\mathbf{Q}_0 \in S_0$, and due to the finiteness of S_0, S_1 the mapping is bijection.

□

We can have an immediate corollary as follows.

Corollary 1. Let $\mathbf{G}_0$ and $\mathbf{G}_1$ be two generator matrices of $\mathcal{M}_n$ -equivalent codes. Define the following two random variables:

$$\begin{aligned} Y_0 &:= (\text{CF}(\mathbf{G}'), \mathbf{Q}\rho_0) \quad \text{where } \mathbf{Q} \leftarrow \$ \mathcal{M}_n, (\mathbf{G}', \rho_0) \leftarrow \text{RREF}^*(\mathbf{G}_0\mathbf{Q}); \\ Y_1 &:= (\text{CF}(\mathbf{G}'), \mathbf{Q}\rho_1) \quad \text{where } \mathbf{Q} \leftarrow \$ \mathcal{M}_n, (\mathbf{G}', \rho_1) \leftarrow \text{RREF}^*(\mathbf{G}_1\mathbf{Q}). \end{aligned}$$

Then Y_0 and Y_1 are identically distributed as random variables over $\mathbb{F}_q^{k \times n} \times \mathcal{M}_n$.

4.2 Compact Sequentially Secure Blind Signature from Linear Equivalence Problem in Canonical Form

Let G represent the monomial group of dimension n over $\mathbb{F}_q$. Let $\alpha \in \mathbb{N}$ and $\kappa = \kappa(\lambda) \geq 2$, $\mu = \mu(\lambda) = \alpha \cdot \kappa(\lambda)$ be polynomial functions in the security parameter λ . Let $H : \{0, 1\}^* \rightarrow [\kappa]^\mu$ be a hash function with the following property: for any string m and $i, j \in [\kappa]$ the number of occurrences of i in $H(m)$ is equal to those of j .

KeyGen(λ): Generate a random systematic matrix $\mathbf{G} \in \mathbb{F}_q^{k \times n}$. Set $\mathbf{Q}_1 = \text{id}$ and $\text{pk}_1 = \mathbf{G}$. Independently sample $\mathbf{Q}_2, \dots, \mathbf{Q}_\kappa \leftarrow G$ and compute $\text{pk}_i = \text{RREF}(\mathbf{G}\mathbf{Q}_i)$ for $i = 2, \dots, \kappa$. Output the secret key $\text{sk} = (\mathbf{Q}_1, \mathbf{Q}_2, \dots, \mathbf{Q}_\kappa)$ and public key $\text{pk} = (\text{pk}_1, \dots, \text{pk}_\kappa)$.

Signer₁(sk): sample group elements $\tilde{\mathbf{Q}}_1, \dots, \tilde{\mathbf{Q}}_\gamma \leftarrow G^\gamma$. For each $i \in [\gamma]$, compute $\mathbf{A}_i \leftarrow \text{RREF}(\mathbf{G}\tilde{\mathbf{Q}}_i)$. Send $Y = (\mathbf{A}_1, \dots, \mathbf{A}_\gamma)$ to User and keep $\tilde{\mathbf{Q}}_1, \dots, \tilde{\mathbf{Q}}_\gamma$.

User₁(pk, Y, m): sample group elements $(\mathbf{R}_1, \dots, \mathbf{R}_\gamma) \leftarrow G^\gamma$ and a random permutation $\pi : [\gamma] \rightarrow [\gamma]$. Compute $(\mathbf{I} \mid \mathbf{B}_i), \rho_i \leftarrow \text{RREF}^*(\mathbf{A}_i \mathbf{R}_i)$, then $\mathbf{B}_i \leftarrow \text{CF}(\mathbf{B}_i)$ for all $i \in [\gamma]$. Compute the challenge $C = H((\mathbf{I} \mid \mathbf{B}_{\pi(1)}), \dots, (\mathbf{I} \mid \mathbf{B}_{\pi(\gamma)}), m)$. Interpret C as a list $(c_1, \dots, c_\gamma)$ and set $C' = (c_{\pi^{-1}(1)}, \dots, c_{\pi^{-1}(\gamma)})$, denoted as $(c'_1, \dots, c'_\gamma)$. Send C' to Signer.

Signer₂(sk, $C', (\tilde{\mathbf{Q}}_1, \dots, \tilde{\mathbf{Q}}_\gamma)$): For each $i \in [\gamma]$ set $\mathbf{Q}'_i = \mathbf{Q}_{c'_i}^{-1} \cdot \tilde{\mathbf{Q}}_i$. Send $S' = (\mathbf{Q}'_1, \dots, \mathbf{Q}'_\gamma)$ to the User.

User₂(pk, $S', \pi, (\mathbf{R}_1, \mathbf{B}_1, \dots, \mathbf{R}_\gamma, \mathbf{B}_\gamma), m$): For each $i \in [\gamma]$, verify that $\text{RREF}(\text{pk}_{c'_i} \mathbf{Q}'_i) \stackrel{?}{=} \mathbf{A}_i$ holds; abort if any check fails. For each $i \in [\gamma]$, compute $\hat{\mathbf{Q}}_i = \mathbf{Q}'_i \cdot \mathbf{R}_i$ and $J_i = \text{CosetRep}(\rho_i \circ \hat{\mathbf{Q}}_i)$. Return $\text{sig} = (C, (J_{\pi(1)}, \dots, J_{\pi(\gamma)}))$.

Verify(pk, m, sig): Parse $\text{sig} = (C = (c_1, \dots, c_\gamma), (J_1, \dots, J_\gamma))$. For each $i \in [\gamma]$, compute $\mathbf{B}_i = \text{CF}((\text{pk}_{c_i})_{J_i}^{-1} \cdot (\text{pk}_{c_i})_{J_i})$. Return 1 if $C \stackrel{?}{=} H((\mathbf{I} \mid \mathbf{B}_1), \dots, (\mathbf{I} \mid \mathbf{B}_\gamma), m)$; otherwise output 0.

Scheme 3: LEAF-2- (κ, γ) : Our improved version of Scheme 1 (Tanuki-2-LESS). One can remove the identity matrices $\mathbf{I}$ in the hash input. The inclusion here is to make the reduction smoother.

Theorem 4 (Correctness). *Scheme 3 has correctness.*

Proof. See the proof of Theorem 7.

Theorem 5 (Blindness). *Scheme 3 has blindness under malicious key.*

Proof. To show the blindness, we have to show that for arbitrary two signatures

$$\text{sig} = ((\mathbf{B}_1, \dots, \mathbf{B}_\gamma), (c_1, \dots, c_\gamma), (J_1, \dots, J_\gamma)),$$

$$\text{sig}' = ((\mathbf{B}'_1, \dots, \mathbf{B}'_\gamma), (c'_1, \dots, c'_\gamma), (J'_1, \dots, J'_\gamma)),$$

corresponding to Signer's first messages

$$(\mathbf{A}_1, \dots, \mathbf{A}_\gamma), (\mathbf{A}'_1, \dots, \mathbf{A}'_\gamma).$$

the adversary cannot distinguish the order of the executions with advantage.

Under the malicious key setting, though the generator matrices are not necessarily $\mathcal{M}_n$ -equivalent, the $\mathcal{M}_n$ -equivalence between $(\mathbf{I} \mid \mathbf{B}_i)$, $(\mathbf{I} \mid \mathbf{B}'_{i'})$ and pk_{c_i} can be guaranteed when $c_i = c'_{i'}$, by the validity of the signatures. Hence, this ensures the existence of the user's randomness such that the signer's first message $(\mathbf{A}_1, \dots, \mathbf{A}_\gamma)$ leads to sig' . Without loss of generality, it suffices to show that the probability generating sig, sig' are the same. This is a direct implication of Corollary 1, and we thereby have perfect blindness.

Theorem 6. *The Scheme 3 is sequentially OMUF (resp ℓ -OMUF) secure if Scheme 1 is sequentially OMUF (resp ℓ -OMUF). The reduction is polynomial time in λ without loss in advantage.*

Proof. Suppose there exists an adversary $\mathcal{A}$ that breaks the OMUF security of Scheme 3. We construct an adversary $\mathcal{B}$ that breaks the OMUF security of Scheme 1 as follows:

1. $\mathcal{B}$ initiates the OMUF experiment for Scheme 1 with the challenger $\mathcal{C}$, receiving $\mathbf{G}$ and pk .
2. $\mathcal{B}$ runs $\mathcal{A}$ on input $(\mathbf{G}, \text{pk})$.
3. To simulate the signing oracle, whenever $\mathcal{A}$ submits a signing query, $\mathcal{B}$ forwards it to $\mathcal{C}$ and relays the resulting message Y to $\mathcal{A}$.
4. Upon receiving a string C' from $\mathcal{A}$ as User_1 's message, $\mathcal{B}$ forwards C' to $\mathcal{C}$ and returns the response S' to $\mathcal{A}$.

After $\mathcal{A}$ outputs signatures $\{(C_i = (c_{i,1}, \dots, c_{i,\gamma}), (J_{i,1}, \dots, J_{i,\gamma}), m_i)\}_{i \in [\ell+1]}$, $\mathcal{B}$ produces valid forgeries as follows:

1. For each $i \in [\ell+1]$ and $j \in [\gamma]$, compute $\mathbf{B}_{i,j}$ such that $\text{CF}((\text{pk}_{c_{i,j}})^{-1}_{J_{i,j}} \cdot (\text{pk}_{c_{i,j}})_{\tilde{J}_{i,j}}) = \mathbf{B}_{i,j}$.
2. For each $j \in [\gamma]$, apply Proposition 3 to solve the linear equivalence problem between $\text{pk}_{c_{i,j}}$ and $(\mathbf{I} \mid \mathbf{B}_{i,j})$, obtaining $\hat{\mathbf{Q}}_{i,j}$.
3. Return the set $\{(C_i = (c_{i,1}, \dots, c_{i,\gamma}), (\hat{\mathbf{Q}}_{i,1}, \dots, \hat{\mathbf{Q}}_{i,\gamma}), m_i)\}_{i \in [\ell+1]}$ as forgeries.

This completes the reduction.

4.3 Compact Concurrently Secure Blind Signature from Linear Equivalence Problem in Canonical Form

We next show our improvement on Scheme 2. As noted in Section 1, Scheme 2 requires two consecutive monomial maps between three codes $\mathcal{C}_1, \mathcal{C}_2, \mathcal{C}_3$. To enable

monomial compression, the codes $\mathcal{C}_2$ and $\mathcal{C}_3$ must be in canonical form. However, directly compressing the monomial in this setting is infeasible, as F -equivalence is generally not preserved after applying a random monomial map. That is, for $\mathcal{C}_2 \sim_F \mathcal{C}'_2$, $\mu(\mathcal{C}_2) \sim_F \mu(\mathcal{C}'_2)$ does not hold for all $\mu \in \mathcal{M}_n$. To overcome this, we introduce a trick where the user records the F -component while computing the canonical form of $\mathcal{C}_2$ from a generator matrix, and later uses this auxiliary information to derive the compressed monomial mapping to $\mathcal{C}_3$. In fact, it suffices to store only the permutation part of the F -element, since the `CosetRep` function does not require the scaling factors.

Formally, except for sharing the same KeyRand subroutine as in Scheme 2, we make the following modifications:

KeyRand(pk): Given $\mathbf{pk} = (\mathbf{pk}_1, \dots, \mathbf{pk}_\kappa)$, sample $\mathbf{T}_1, \dots, \mathbf{T}_\kappa \leftarrow \$ G$ and a permutation $\sigma \leftarrow \$ S_\kappa$. For each $i \in [\kappa]$, set $\mathbf{pk}'_i = \text{RREF}(\mathbf{pk}_{\sigma(i)} \mathbf{T}_{\sigma(i)})$. Output $(\mathbf{pk}', (\mathbf{T}_1, \dots, \mathbf{T}_\kappa), \sigma)$, where $\mathbf{pk}' = (\mathbf{pk}'_1, \dots, \mathbf{pk}'_\kappa)$.

we further introduce the following algorithms that adaptively modify their input:

CF*(A): On input a $k \times (n-k)$ matrix $\mathbf{A}$ or say a systematic generator matrix $(\mathbf{I} \mid \mathbf{A})$, return $\text{CF}(\mathbf{A})$ and a monomial $\mu \in F \subseteq \mathcal{M}_n$ such that

$$\text{RREF}(\mu \circ (\mathbf{I} \mid \mathbf{A})) = (\mathbf{I} \mid \text{CF}(\mathbf{A})).$$

KeyRand*(pk): Given $\mathbf{pk} = (\mathbf{pk}_1, \dots, \mathbf{pk}_\kappa)$, sample $\mathbf{T}_1, \dots, \mathbf{T}_\kappa \leftarrow \$ G$ and a permutation $\sigma \leftarrow \$ S_\kappa$. For each $i \in [\kappa]$, set

$$\mathbf{pk}'_i, \rho_i \leftarrow \text{RREF}^*(\mathbf{pk}_{\sigma(i)} \mathbf{T}_{\sigma(i)}),$$

update $\mathbf{T}_{\sigma(i)} \leftarrow \rho_i \circ \mathbf{T}_{\sigma(i)}$, and then set $\mathbf{pk}'_i, \mu \leftarrow \text{CF}^*(\mathbf{pk}'_i)$ and update $\mathbf{T}_{\sigma(i)} \leftarrow \mu \circ \mathbf{T}_{\sigma(i)}$. Output $(\mathbf{pk}', (\mathbf{T}_1, \dots, \mathbf{T}_\kappa), \sigma)$, where $\mathbf{pk}' = (\mathbf{pk}'_1, \dots, \mathbf{pk}'_\kappa)$.

Remark 1. CF^* is computationally feasible in our setting. Indeed, one can apply the polynomial-time reduction from Proposition 3 for F -equivalence to recover the corresponding monomial map. A more efficient method is to trace the execution of CF directly. For the case $F = \mathcal{M}_k \oplus \mathcal{M}_{n-k}$, the computation consists of a sequence of column- and row-sorting and scaling operations [30], which are straightforward to track. In this setting, the augmented version CF^* introduces less than 1% overhead compared to CF .

Let q be a prime, $G = \mathcal{M}_n$ denote the group of $n \times n$ monomial matrices over $\mathbb{F}_q$. Set polynomially-bounded parameters $\kappa \geq 2$, $\alpha > 0$, and $\gamma = \alpha\kappa$ as in the framework. Let $H : \{0, 1\}^* \rightarrow [\kappa]^\gamma$ be a hash function with balanced output, i.e., each symbol in $[\kappa]$ appears equally often.

KeyGen(λ): Sample a random full-rank matrix $\mathbf{G} \in \mathbb{F}_q^{k \times n}$. Let $\mathbf{Q}_1 = \text{id}$ and set $\text{pk}_1 = \text{RREF}(\mathbf{G})$. For $i = 2$ to κ , sample $\mathbf{Q}_i \leftarrow \$ G$ and set $\text{pk}_i = \text{RREF}(\mathbf{G}\mathbf{Q}_i)$. Output $\text{sk} = (\mathbf{Q}_2, \dots, \mathbf{Q}_\kappa)$ and $\text{pk} = (\text{pk}_1, \dots, \text{pk}_\kappa)$.

Signer₁(sk, pk): Run $(\text{pk}', T', \sigma') \leftarrow \text{KeyRand}(\text{pk})$ with $T' = (\mathbf{T}'_1, \dots, \mathbf{T}'_\kappa)$. For each $i \in [\kappa]$, set $\mathbf{Q}'_i = \mathbf{Q}_{\sigma'(i)} \mathbf{T}'_{\sigma'(i)}$. Sample $\tilde{\mathbf{Q}}_1, \dots, \tilde{\mathbf{Q}}_\gamma \leftarrow \$ G$ and compute $\mathbf{A}_i = \text{RREF}(\mathbf{G}\tilde{\mathbf{Q}}_i)$ for $i \in [\gamma]$. Send $(\text{pk}', (\mathbf{A}_1, \dots, \mathbf{A}_\gamma))$ to the user and keep $\tilde{\mathbf{Q}}_1, \dots, \tilde{\mathbf{Q}}_\gamma, \mathbf{Q}'_1, \dots, \mathbf{Q}'_\kappa, T', \sigma'$ as secret state.

User₁(pk', (A₁, ..., A_γ), m): Run $(\text{pk}'', T, \sigma) \leftarrow \text{KeyRand}^*(\text{pk}')$ with $T = (\mathbf{T}_1, \dots, \mathbf{T}_\kappa)$. Sample $\mathbf{R}_1, \dots, \mathbf{R}_\gamma \leftarrow \$ G$ and a random permutation π . Compute $(\mathbf{I} \mid \mathbf{B}_i), \rho_i \leftarrow \text{RREF}^*(\mathbf{A}_i \mathbf{R}_i)$, then $\mathbf{B}_i \leftarrow \text{CF}(\mathbf{B}_i)$ for each $i \in [\gamma]$. $C = H((\mathbf{I} \mid \mathbf{B}_{\pi(1)}), \dots, (\mathbf{I} \mid \mathbf{B}_{\pi(\gamma)}), \text{pk}'', m)$ and parse $C = (c_1, \dots, c_\gamma)$. For each $i \in [\gamma]$, set $c'_i = \sigma(c_{\pi^{-1}(i)})$. Send $C' = (c'_1, \dots, c'_\gamma)$ to the signer.

Signer₂(c', Q̃₁, ..., Q̃_γ, T', σ'): For each $i \in [\gamma]$, compute $\mathbf{S}'_i = (\mathbf{Q}'_{c'_i})^{-1} \tilde{\mathbf{Q}}_i$. Return $(S' = (\mathbf{S}'_1, \dots, \mathbf{S}'_\gamma), T', \sigma')$ to the user.

User₂(pk, S', π, (R₁, ..., R_γ), T, σ, T', σ', m): Verify if $\text{KeyRand}(\text{pk}; T', \sigma') \stackrel{?}{=} \text{pk}'$; abort if not. For each $i \in [\gamma]$, verify that $\text{RREF}(\text{pk}'_{c'_i} \cdot \mathbf{S}'_i) \stackrel{?}{=} \mathbf{A}_i$; abort if not. For each $i \in [\gamma]$, set $J_i = \text{CosetRep}(\rho_i \circ \mathbf{T}_{c'_i}^{-1} \mathbf{S}'_i \mathbf{R}_i)$. For each $j \in [\kappa]$, compute $I_j = \text{CosetRep}(\mathbf{T}'_{\sigma'(\sigma(j))} \mathbf{T}_{\sigma(j)})$. Let $\tau = \sigma' \circ \sigma$ and return

$$\text{sig} = (C, (J_{\pi(1)}, \dots, J_{\pi(\gamma)}), (I_1, \dots, I_\kappa), \tau).$$

Verify(pk, m, sig): Parse $\text{sig} = (C = (c_1, \dots, c_\gamma), (J_1, \dots, J_\gamma), (I_1, \dots, I_\kappa), \tau)$. For each $i \in [\kappa]$, compute $\overline{\text{pk}}_i = (\mathbf{I} \mid \text{CF}((\text{pk}_{\tau(i)})_{I_i}^{-1} (\text{pk}_{\tau(i)})_{\bar{I}_i}))$. For each $i \in [\gamma]$, compute $\mathbf{B}_i = \text{CF}((\overline{\text{pk}}_{c_i})_{J_i}^{-1} (\overline{\text{pk}}_{c_i})_{\bar{J}_i})$. Return 1 if $C \stackrel{?}{=} H((\mathbf{I} \mid \mathbf{B}_1), \dots, (\mathbf{I} \mid \mathbf{B}_\gamma), \overline{\text{pk}}, m)$ holds; otherwise output 0.

Scheme 4: LEAF-3-(κ, γ) : Our improved version of Scheme 2 (Tanuki-3-LESS). One can remove the identity matrices $\mathbf{I}$ in the hash input. The inclusion here is to make the reduction smoother.

Theorem 7 (Correctness). *Scheme 4 has correctness.*

Proof. We begin by showing that the user does not abort in the **User₂** step when both parties behave honestly.

Fix any round $i \in [\gamma]$. The signer samples $\tilde{\mathbf{Q}}_i \leftarrow \$ G$ and computes

$$\mathbf{A}_i = \text{RREF}(\mathbf{G}\tilde{\mathbf{Q}}_i).$$

At Signer_2 , the value returned is

$$\mathbf{S}'_i = (\mathbf{Q}'_{c'_i})^{-1} \cdot \tilde{\mathbf{Q}}_i, \quad \text{where } \mathbf{Q}'_{c'_i} = \mathbf{Q}_{\sigma'(c'_i)} \cdot \mathbf{T}'_{\sigma'(c'_i)}.$$

Now consider $\text{pk}'_{c'_i} = \text{RREF}(\mathbf{G}\mathbf{Q}'_{c'_i})$. Then:

$$\text{RREF}(\text{pk}'_{c'_i} \cdot \mathbf{S}'_i) = \text{RREF}(\text{RREF}(\mathbf{G}\mathbf{Q}'_{c'_i}) \cdot (\mathbf{Q}'_{c'_i})^{-1} \cdot \tilde{\mathbf{Q}}_i) = \text{RREF}(\mathbf{G}\tilde{\mathbf{Q}}_i) = \mathbf{A}_i.$$

Hence, User_2 does not abort.

It suffices to show Verify will return 1.

Recall that for every $j \in [\kappa]$ the user computes $I_j = \text{CosetRep}(\mathbf{T}'_{\sigma'(j)} \mathbf{T}_{\sigma(j)})$ and $\tau = \sigma' \circ \sigma$. Since $\text{RREF}(\text{pk}_{\tau(j)} \mathbf{T}'_{\sigma'(j)} \mathbf{T}_{\sigma(j)}) = \text{RREF}(\text{pk}'_{\sigma(j)} \mathbf{T}_{\sigma(j)})$ is systematic due to the mechanism of KeyRand^* , by Lemma 1 we have

$$\overline{\text{pk}}_j = (\mathbf{I} \mid \text{CF}((\text{pk}_{\tau(j)})_{I_j}^{-1} (\text{pk}_{\tau(j)})_{I_j})).$$

Hence the generator matrices $\overline{\text{pk}}_j$, $\text{pk}'_{\sigma(j)} \mathbf{T}_{\sigma(j)}$ and $\text{pk}_{\tau(j)} \mathbf{T}'_{\sigma'(j)} \mathbf{T}_{\sigma(j)}$ are F -equivalent due to the update made in CF^* .

For each round $i \in [\gamma]$, denote c_i the i -th coordinate of C and recall $J_i = \text{CosetRep}(\rho_i \circ \hat{\mathbf{Q}}_i)$ with $\hat{\mathbf{Q}}_i = \mathbf{T}_{c'_i}^{-1} \mathbf{S}'_i \mathbf{R}_i$.

$$\begin{aligned} \overline{\text{pk}}_{c_i} \hat{\mathbf{Q}}_{\pi(i)} &\sim_F \text{pk}'_{\sigma(c_i)} \mathbf{T}_{\sigma(c_i)} \hat{\mathbf{Q}}_{\pi(i)} \\ &\sim_F \text{pk}'_{\sigma(c_i)} \mathbf{T}_{\sigma(c_i)} \mathbf{T}_{\sigma(c_i)}^{-1} \mathbf{S}'_{\pi(i)} \mathbf{R}_{\pi(i)} \\ &\sim_F \text{pk}'_{\sigma(c_i)} \mathbf{S}'_{\pi(i)} \mathbf{R}_{\pi(i)} \\ &\sim_F \mathbf{A}_{\pi(i)} \mathbf{R}_{\pi(i)}, \end{aligned} \tag{User2}$$

where the last term will be systematic after applied by ρ_i and be $\mathbf{B}_{\pi(i)}$ as computed in User_1 . By Lemma 1, Verify will return 1.

Theorem 8 (Blindness). *Scheme 4 has blindness under malicious key.*

Proof. To show the blindness, we have to show that for arbitrary two signatures

$$\text{sig} = (\overline{\text{pk}}, (\mathbf{B}_1, \dots, \mathbf{B}_\gamma), (c_1, \dots, c_\gamma), (J_1, \dots, J_\gamma), (I_1, \dots, I_\kappa), \tau),$$

$$\text{sig}' = (\overline{\text{pk}}', (\mathbf{B}'_1, \dots, \mathbf{B}'_\gamma), (c'_1, \dots, c'_\gamma), (J'_1, \dots, J'_\gamma), (I'_1, \dots, I'_\kappa), \tau'),$$

corresponding to Signer's first messages

$$((\widetilde{\text{pk}}, \mathbf{A}_1, \dots, \mathbf{A}_\gamma), ((\widetilde{\text{pk}}', \mathbf{A}'_1, \dots, \mathbf{A}'_\gamma)).$$

the adversary cannot distinguish the order of the executions with advantage. Here, $\overline{\text{pk}}, \overline{\text{pk}}'$ are computed from $((I_1, \dots, I_\kappa), \tau), ((I'_1, \dots, I'_\kappa), \tau')$ respectively. Also, $(\mathbf{B}_1, \dots, \mathbf{B}_\gamma), (\mathbf{B}'_1, \dots, \mathbf{B}'_\gamma)$ are computed from $((c_1, \dots, c_\gamma), (J_1, \dots, J_\gamma))$ and $((c'_1, \dots, c'_\gamma), (J'_1, \dots, J'_\gamma))$ respectively.

For the session specific key $\widetilde{\mathbf{pk}}$, by Corollary 1, the probabilities of resulting in $\overline{\mathbf{pk}}, \overline{\mathbf{pk}}'$ are the same regardless of the update made in CF^* in KeyRand^* . The same applies for $\widetilde{\mathbf{pk}}'$.

Under the malicious key setting, though the generator matrices are not necessarily $\mathcal{M}_n$ -equivalent, the $\mathcal{M}_n$ -equivalence between $(\mathbf{I} \mid \mathbf{B}_i)$, $(\mathbf{I} \mid \mathbf{B}'_{i'})$ and $\overline{\mathbf{pk}}_{c_i}$ can be guaranteed when $c_i = c'_{i'}$ by the validness of the signatures. Hence, this ensures the existence of the user's randomness such that the signer's first message $(\mathbf{A}_1, \dots, \mathbf{A}_\gamma)$ leads to sig' . For any first message $\tilde{C}$ of signer, we have $\Pr_\pi[\pi(\tilde{C}) = (c_1, \dots, c_\gamma)] = \Pr_\pi[\pi(\tilde{C}) = (c'_1, \dots, c'_\gamma)]$ where π is drawn uniformly at random. Without loss of generality, it suffices to show that the probability generating sig, sig' are the same. This is a direct implication of Corollary 1, and we thereby have perfect blindness.

Theorem 9. *Scheme 4 is ℓ -OMUF secure if Scheme 2 is ℓ -OMUF secure. The reduction is polynomial time in λ without loss in advantage.*

Proof. Suppose there exists an adversary $\mathcal{A}$ that breaks the OMUF security of Scheme 4. We construct an adversary $\mathcal{B}$ that breaks the OMUF security of Scheme 2 as follows:

1. $\mathcal{B}$ initiates the OMUF experiment for Scheme 1 with the challenger $\mathcal{C}$, receiving $\mathbf{G}$ and $\mathbf{pk}$.
2. $\mathcal{B}$ runs $\mathcal{A}$ on input $(\mathbf{G}, \mathbf{pk})$.
3. To simulate the signing oracle, whenever $\mathcal{A}$ submits a signing query, $\mathcal{B}$ forwards it to $\mathcal{C}$ and relays the resulting message Y to $\mathcal{A}$.
4. Upon receiving a string C' from $\mathcal{A}$ as User_1 's message, $\mathcal{B}$ forwards C' to $\mathcal{C}$ and returns the response S' to $\mathcal{A}$.

After $\mathcal{A}$ outputs signatures

$$\{(C_i = (c_{i,1}, \dots, c_{i,\gamma}), (J_{i,1}, \dots, J_{i,\gamma}), (I_{i,1}, \dots, I_{i,\kappa}), \tau_i, m_i)\}_{i \in [\ell+1]},$$

$\mathcal{B}$ produces valid forgeries as follows:

1. For each $i \in [\ell+1]$ and $j \in [\kappa]$, $\overline{\mathbf{pk}}_{i,j} = \left(\mathbf{I} \mid \text{CF} \left((\mathbf{pk}_{\tau(i)})_{I_{i,j}}^{-1} (\mathbf{pk}_{\tau(i)})_{\bar{I}_{i,j}} \right) \right)$.
2. For each $i \in [\ell+1]$ and $j \in [\gamma]$, compute $\mathbf{B}_{i,j}$ such that $\text{CF}((\overline{\mathbf{pk}}_{c_{i,j}})_{J_{i,j}}^{-1} \cdot (\overline{\mathbf{pk}}_{c_{i,j}})_{\bar{J}_{i,j}}) = \mathbf{B}_{i,j}$.
3. For each $i \in [\ell+1]$ and each $j \in [\gamma]$, apply Proposition 3 to solve the linear equivalence problem between $\overline{\mathbf{pk}}_{c_{i,j}}$ and $(\mathbf{I} \mid \mathbf{B}_{i,j})$, obtaining $\hat{\mathbf{Q}}_{i,j}$.
4. For each $i \in [\ell+1]$ and $j \in [\kappa]$, again, apply Proposition 3 to solve the linear equivalence problem between $\overline{\mathbf{pk}}_{i,j}$ and $\mathbf{pk}_{i,\tau_i(j)}$ obtaining $\hat{\mathbf{T}}_{i,\tau_i(j)}$.
5. Return the set

$$\{(C_i = (c_{i,1}, \dots, c_{i,\gamma}), (\hat{\mathbf{Q}}_{i,1}, \dots, \hat{\mathbf{Q}}_{i,\gamma}), (\hat{\mathbf{T}}_{i,1}, \dots, \hat{\mathbf{T}}_{i,\kappa}), m_i)\}_{i \in [\ell+1]}$$

as forgeries.

The use of Proposition 3 gives the corresponding relations needed in the verification of Scheme 2, and hence gives valid forgeries.

	sk	pk	sig	Seq.OMUF	Con.OMUF	Paradigm
Tanuki-2-LESS-(16,80)	512 B	204 KB	36.9 KB	✓	$\log(\lambda)$	Σ -Protocol
Tanuki-3-LESS-(54,108)	1.66 KB	530 KB	74.9 KB	✓	$\text{poly}(\lambda)$	Σ -Protocol
Tanuki-3-LESS-(70,70)	2.16 KB	530 KB	64.7 KB	✓	$\text{poly}(\lambda)$	Σ -Protocol
CSI-Otter- ζ_4 [43]	16 B	512 B	6* KB	✓	$\log(\lambda)$	Σ -Protocol
[1]	- KB	- KB	45 KB	✓	$\text{poly}(\lambda)$	Fischlin
[51]	- KB	- KB	102 KB	✓	$\text{poly}(\lambda)$	Fischlin
[16]	2 KB	1 KB	22 KB	✓	$\text{poly}(\lambda)$	Fischlin
LEAF-2-(16,80)	512 B	204 KB	2.5 KB	✓	$\log(\lambda)$	Σ -Protocol
LEAF-3-(54,108)	1.63 KB	719 KB	5.10 KB	✓	$\text{poly}(\lambda)$	Σ -Protocol
LEAF-3-(70,70)	2.16 KB	936 KB	4.42 KB	✓	$\text{poly}(\lambda)$	Σ -Protocol

Table 1. Comparison of key and signature sizes of different post-quantum at the 128-bit security level (i.e. $\lambda = 128$). The columns “Seq.OMUF & Con.OMUF” indicate the sequential security and concurrent security respectively. The former notion means OMUF security by allowing only one execution at the same time. While the x-concurrent notion consider OMUF allow x signing session at the same time. For the performance of CSI-Otter [43], we scale their number of repetitions by 1.5 because the original number does not offer sufficient sequential security as mentioned in [33,44].

5 Performance

We provide a C implementation on an Intel Core i7-12700H @ 2.30 GHz, combining and adapting the AVX2-optimized LESS implementation of CF and RREF functions from [6] into our specification using the flags `-O3 -mavx2 -mavx -march=native -mtune=native -ftree-vectorize`. Table 1 gives a size comparison with other post-quantum blind signatures, and Table 2 shows the corresponding runtime.

References

1. Agrawal, S., Kirshanova, E., Stehlé, D., Yadav, A.: Practical, round-optimal lattice-based blind signatures. In: Yin, H., Stavrou, A., Cremers, C., Shi, E. (eds.) ACM CCS 2022: 29th Conference on Computer and Communications Security. pp. 39–53. ACM Press (Nov 2022). <https://doi.org/10.1145/3548606.3560650>
2. Alkeilani Alkadri, N., El Bansarkhani, R., Buchmann, J.: On lattice-based interactive protocols: An approach with less or no aborts. In: Liu, J.K., Cui, H. (eds.) ACISP 20: 25th Australasian Conference on Information Security and Privacy. Lecture Notes in Computer Science, vol. 12248, pp. 41–61. Springer, Cham (Nov / Dec 2020). https://doi.org/10.1007/978-3-030-55304-3_3
3. Alkeilani Alkadri, N., Harasser, P., Janson, C.: BlindOR: an efficient lattice-based blind signature scheme from OR-proofs. In: Conti, M., Stevens, M., Krenn, S. (eds.) CANS 21: 20th International Conference on Cryptology and Network Security. Lecture Notes in Computer Science, vol. 13099, pp. 95–115. Springer, Cham (Dec 2021). https://doi.org/10.1007/978-3-030-92548-2_6
4. Apple Inc.: icloud private relay overview. https://www.apple.com/icloud/docs/iCloud_Private_Relay_Overview_Dec2021.pdf (2021), accessed: 2025-08-05

Scheme	(x = 2; $\kappa = 16$, $\mu = 80$)			(x = 3; $\kappa = 54$, $\mu = 108$)			(x = 3; $\kappa = 70$, $\mu = 70$)		
	KeyGen	Sign	Verify	KeyGen	Sign	Verify	KeyGen	Sign	Verify
Runtime in ms									
Tanuki-LESS-x	2.81	47.2	16.1	9.49	94.8	32.8	12.7	81.0	28.0
LEAF-x	2.76	57.2	25.8	10.3	117	54.8	12.7	99.2	47.5
Runtime in Mcycles									
Tanuki-LESS-x	8.13	126	43.6	26.7	252	87.6	35.1	218	75.6
LEAF-x	7.49	153	69.9	26.7	316	147	33.9	268	127
Slowdown Factor in Cycles (LEAF / Tanuki)									
	-	1.21×	1.6×	-	1.25×	1.7×	-	1.22×	1.69×

Table 2. Comparison of Tanuki-LESS and LEAF runtimes on an Intel Core i7-12700H @ 2.30 GHz using the same flags on average of 1000 executions respectively. The slowdown ratio shows how many times slower LEAF is compared to LESS. As the key generation algorithms are identical, it is pointless to compare the costs.

5. Apple Inc.: Private cloud compute: A new frontier for ai privacy in the cloud. <https://security.apple.com/blog/private-cloud-compute/> (2024), accessed: 2025-08-05
6. Baldi, M., Barenghi, A., Beckwith, L., Biasse, J., Chou, T., Esser, A., Gaj, K., Karl, P., Mohajerani, K., Pelosi, G., Persichetti, E., Saarinen, M.O., Santini, P., Wallace, R., Zwydinger, F.: LESS — Linear Equivalence Signature Scheme. Tech. rep., National Institute of Standards and Technology (2024), available at <https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures>
7. Baldi, M., Barenghi, A., Beckwith, L., Biasse, J., Esser, A., Gaj, K., Mohajerani, K., Pelosi, G., Persichetti, E., Saarinen, M.O., Santini, P., Wallace, R.: LESS — Linear Equivalence Signature Scheme. Tech. rep., National Institute of Standards and Technology (2023), available at <https://csrc.nist.gov/Projects/pqc-dig-sig/round-1-additional-signatures>
8. Barenghi, A., Biasse, J.F., Ngo, T., Persichetti, E., Santini, P.: Advanced signature functionalities from the code equivalence problem. Cryptology ePrint Archive, Report 2022/710 (2022), <https://eprint.iacr.org/2022/710>
9. Battagliola, M., Borin, G., Di Crescenzo, G., Meneghetti, A., Persichetti, E.: Enhancing threshold group action signature schemes: Adaptive security and scalability improvements. Cryptology ePrint Archive, Report 2025/085 (2025), <https://eprint.iacr.org/2025/085>
10. Battagliola, M., Borin, G., Meneghetti, A., Persichetti, E.: Cutting the GRASS: Threshold GRoup action signature schemes. In: Oswald, E. (ed.) Topics in Cryptology – CT-RSA 2024. Lecture Notes in Computer Science, vol. 14643, pp. 460–489. Springer, Cham (May 2024). https://doi.org/10.1007/978-3-031-58868-6_18
11. Beckwith, L., Esser, A., Persichetti, E., Santini, P., Zwydinger, F.: LESS is even more: Optimizing digital signatures from code equivalence. Cryptology ePrint Archive, Paper 2025/1424 (2025), <https://eprint.iacr.org/2025/1424>
12. Benhamouda, F., Lepoint, T., Loss, J., Orrù, M., Raykova, M.: On the (in)security of ROS. In: Canteaut, A., Standaert, F.X. (eds.) Advances in Cryptology – EURO-CRYPT 2021, Part I. Lecture Notes in Computer Science, vol. 12696, pp. 33–53. Springer, Cham (Oct 2021). https://doi.org/10.1007/978-3-030-77870-5_2

13. Beullens, W., Dobson, S., Katsumata, S., Lai, Y.F., Pintore, F.: Group signatures and more from isogenies and lattices: Generic, simple, and efficient. In: Dunkelman, O., Dziembowski, S. (eds.) *Advances in Cryptology – EUROCRYPT 2022*, Part II. *Lecture Notes in Computer Science*, vol. 13276, pp. 95–126. Springer, Cham (May / Jun 2022). https://doi.org/10.1007/978-3-031-07085-3_4
14. Beullens, W., Katsumata, S., Pintore, F.: Calamari and Falaf: Logarithmic (linkable) ring signatures from isogenies and lattices. In: Moriai, S., Wang, H. (eds.) *Advances in Cryptology – ASIACRYPT 2020*, Part II. *Lecture Notes in Computer Science*, vol. 12492, pp. 464–492. Springer, Cham (Dec 2020). https://doi.org/10.1007/978-3-030-64834-3_16
15. Beullens, W., Kleinjung, T., Vercauteren, F.: CSI-FiSh: Efficient isogeny based signatures through class group computations. In: Galbraith, S.D., Moriai, S. (eds.) *Advances in Cryptology – ASIACRYPT 2019*, Part I. *Lecture Notes in Computer Science*, vol. 11921, pp. 227–247. Springer, Cham (Dec 2019). https://doi.org/10.1007/978-3-030-34578-5_9
16. Beullens, W., Lyubashevsky, V., Nguyen, N.K., Seiler, G.: Lattice-based blind signatures: Short, efficient, and round-optimal. In: Meng, W., Jensen, C.D., Cremers, C., Kirda, E. (eds.) *ACM CCS 2023: 30th Conference on Computer and Communications Security*. pp. 16–29. ACM Press (Nov 2023). <https://doi.org/10.1145/3576915.3616613>
17. BIASSE, J.F., MICHELI, G., PERSICETTI, E., SANTINI, P.: LESS is more: Code-based signatures without syndromes. In: Nitaj, A., Youssef, A.M. (eds.) *AFRICACRYPT 20: 12th International Conference on Cryptology in Africa*. *Lecture Notes in Computer Science*, vol. 12174, pp. 45–65. Springer, Cham (Jul 2020). https://doi.org/10.1007/978-3-030-51938-4_3
18. BIS Innovation Lab.: Project tourbillon - exploring privacy, security and scalability for cbdcs - final report. <https://www.bis.org/publ/othp80.pdf> (2023)
19. Bläser, M., Duong, D.H., Narayanan, A.K., Plantard, T., Qiao, Y., Sipasseuth, A., Tang, G.: ALTEQ. Tech. rep., National Institute of Standards and Technology (2023), available at <https://csrc.nist.gov/Projects/pqc-dig-sig/round-1-additional-signatures>
20. Boldyreva, A.: Threshold signatures, multisignatures and blind signatures based on the gap-Diffie-Hellman-group signature scheme. In: Desmedt, Y. (ed.) *PKC 2003: 6th International Workshop on Theory and Practice in Public Key Cryptography*. *Lecture Notes in Computer Science*, vol. 2567, pp. 31–46. Springer, Berlin, Heidelberg (Jan 2003). https://doi.org/10.1007/3-540-36288-6_3
21. Bootle, J., Lyubashevsky, V., Seiler, G.: Algebraic techniques for short(er) exact lattice-based zero-knowledge proofs. In: Boldyreva, A., Micciancio, D. (eds.) *Advances in Cryptology – CRYPTO 2019*, Part I. *Lecture Notes in Computer Science*, vol. 11692, pp. 176–202. Springer, Cham (Aug 2019). https://doi.org/10.1007/978-3-030-26948-7_7
22. Bouillaguet, C., Feneuil, T., Maire, J., Rivain, M., Sauvage, J., Vergnaud, D.: Blinding post-quantum hash-and-sign signatures. *Cryptology ePrint Archive*, Paper 2025/895 (2025), <https://eprint.iacr.org/2025/895>
23. Budroni, A., Chi-Domínguez, J.J., D’Alconzo, G., Scala, A.J.D., Kulkarni, M.: Don’t use it twice! Solving relaxed linear equivalence problems. In: *Advances in Cryptology – ASIACRYPT 2024*, Part VIII. pp. 35–65. *Lecture Notes in Computer Science*, Springer, Singapore (Dec 2024). https://doi.org/10.1007/978-981-96-0944-4_2

24. Budroni, A., Chi-Domínguez, J.J., Franch, E.: Don't use it twice: Reloaded! On the lattice isomorphism group action. Cryptology ePrint Archive, Report 2025/516 (2025), <https://eprint.iacr.org/2025/516>
25. Castryck, W., Lange, T., Martindale, C., Panny, L., Renes, J.: CSIDH: An efficient post-quantum commutative group action. In: Peyrin, T., Galbraith, S. (eds.) *Advances in Cryptology – ASIACRYPT 2018, Part III*. Lecture Notes in Computer Science, vol. 11274, pp. 395–427. Springer, Cham (Dec 2018). https://doi.org/10.1007/978-3-030-03332-3_15
26. Chairattana-Apirom, R., Hanzlik, L., Loss, J., Lysyanskaya, A., Wagner, B.: PI-cut-choo and friends: Compact blind signatures via parallel instance cut-and-choose and more. In: Dodis, Y., Shrimpton, T. (eds.) *Advances in Cryptology – CRYPTO 2022, Part III*. Lecture Notes in Computer Science, vol. 13509, pp. 3–31. Springer, Cham (Aug 2022). https://doi.org/10.1007/978-3-031-15982-4_1
27. Chairattana-Apirom, R., Tessaro, S., Zhu, C.: Pairing-free blind signatures from CDH assumptions. In: *Advances in Cryptology – CRYPTO 2024, Part I*. pp. 174–209. Lecture Notes in Computer Science, Springer, Cham (Aug 2024). https://doi.org/10.1007/978-3-031-68376-3_6
28. Chaum, D.: Blind signatures for untraceable payments. In: Chaum, D., Rivest, R.L., Sherman, A.T. (eds.) *Advances in Cryptology – CRYPTO'82*. pp. 199–203. Plenum Press, New York, USA (1982). https://doi.org/10.1007/978-1-4757-0602-4_18
29. Chou, T., Niederhagen, R., Persichetti, E., Ran, L., Randrianarisoa, T.H., Reijnders, K., Samardjiska, S., Trimoska, M.: MEDS — Matrix Equivalence Digital Signature. Tech. rep., National Institute of Standards and Technology (2023), available at <https://csrc.nist.gov/Projects/pqc-dig-sig/round-1-additional-signatures>
30. Chou, T., Persichetti, E., Santini, P.: On linear equivalence, canonical forms, and digital signatures. Cryptology ePrint Archive, Report 2023/1533 (2023), <https://eprint.iacr.org/2023/1533>
31. Davidson, A., Goldberg, I., Sullivan, N., Tankersley, G., Valsorda, F.: Privacy pass: Bypassing internet challenges anonymously. *Proceedings on Privacy Enhancing Technologies* **2018**(3), 164–180 (Jul 2018). <https://doi.org/10.1515/popets-2018-0026>
32. De Feo, L., Meyer, M.: Threshold schemes from isogeny assumptions. In: Kiayias, A., Kohlweiss, M., Wallden, P., Zikas, V. (eds.) *PKC 2020: 23rd International Conference on Theory and Practice of Public Key Cryptography, Part II*. Lecture Notes in Computer Science, vol. 12111, pp. 187–212. Springer, Cham (May 2020). https://doi.org/10.1007/978-3-030-45388-6_7
33. Do, K., Hanzlik, L., Paracucchi, E.: M&M'S: Mix and match attacks on Schnorr-type blind signatures with repetition. In: Joye, M., Leander, G. (eds.) *Advances in Cryptology – EUROCRYPT 2024, Part VI*. Lecture Notes in Computer Science, vol. 14656, pp. 363–387. Springer, Cham (May 2024). https://doi.org/10.1007/978-3-031-58751-1_13
34. Fischlin, M.: Round-optimal composable blind signatures in the common reference string model. In: Dwork, C. (ed.) *Advances in Cryptology – CRYPTO 2006*. Lecture Notes in Computer Science, vol. 4117, pp. 60–77. Springer, Berlin, Heidelberg (Aug 2006). https://doi.org/10.1007/11818175_4
35. Fuchsbauer, G., Hanser, C., Kamath, C., Slamanig, D.: Practical round-optimal blind signatures in the standard model from weaker assumptions. In: Zikas, V., De Prisco, R. (eds.) *SCN 16: 10th International Conference on Security in Com-*

- communication Networks. Lecture Notes in Computer Science, vol. 9841, pp. 391–408. Springer, Cham (Aug / Sep 2016). https://doi.org/10.1007/978-3-319-44618-9_21
36. Hanzlik, L., Lai, Y.F., Mula, M., Paracucchi, E., Slamanig, D., Tang, G.: Tanuki: New frameworks for (concurrently secure) blind signatures from post-quantum groups actions. Cryptology ePrint Archive, Paper 2025/1100 (2025), <https://eprint.iacr.org/2025/1100>
37. Hanzlik, L., Loss, J., Wagner, B.: Rai-choo! Evolving blind signatures to the next level. In: Hazay, C., Stam, M. (eds.) Advances in Cryptology – EUROCRYPT 2023, Part V. Lecture Notes in Computer Science, vol. 14008, pp. 753–783. Springer, Cham (Apr 2023). https://doi.org/10.1007/978-3-031-30589-4_26
38. Hauck, E., Kiltz, E., Loss, J.: A modular treatment of blind signatures from identification schemes. In: Ishai, Y., Rijmen, V. (eds.) Advances in Cryptology – EUROCRYPT 2019, Part III. Lecture Notes in Computer Science, vol. 11478, pp. 345–375. Springer, Cham (May 2019). https://doi.org/10.1007/978-3-030-17659-4_12
39. Hauck, E., Kiltz, E., Loss, J., Nguyen, N.K.: Lattice-based blind signatures, revisited. In: Micciancio, D., Ristenpart, T. (eds.) Advances in Cryptology – CRYPTO 2020, Part II. Lecture Notes in Computer Science, vol. 12171, pp. 500–529. Springer, Cham (Aug 2020). https://doi.org/10.1007/978-3-030-56880-1_18
40. Heilman, E., Alshenibr, L., Baldimtsi, F., Scafuro, A., Goldberg, S.: TumbleBit: An untrusted bitcoin-compatible anonymous payment hub. In: ISOC Network and Distributed System Security Symposium – NDSS 2017. The Internet Society (Feb / Mar 2017). <https://doi.org/10.14722/ndss.2017.23086>
41. Kastner, J., Loss, J., Xu, J.: The Abe-Okamoto partially blind signature scheme revisited. In: Agrawal, S., Lin, D. (eds.) Advances in Cryptology – ASIACRYPT 2022, Part IV. Lecture Notes in Computer Science, vol. 13794, pp. 279–309. Springer, Cham (Dec 2022). https://doi.org/10.1007/978-3-031-22972-5_10
42. Kastner, J., Loss, J., Xu, J.: On pairing-free blind signature schemes in the algebraic group model. In: Hanaoka, G., Shikata, J., Watanabe, Y. (eds.) PKC 2022: 25th International Conference on Theory and Practice of Public Key Cryptography, Part II. Lecture Notes in Computer Science, vol. 13178, pp. 468–497. Springer, Cham (Mar 2022). https://doi.org/10.1007/978-3-030-97131-1_16
43. Katsumata, S., Lai, Y.F., LeGrow, J.T., Qin, L.: CSI-Otter: Isogeny-based (partially) blind signatures from the class group action with a twist. In: Handschuh, H., Lysyanskaya, A. (eds.) Advances in Cryptology – CRYPTO 2023, Part III. Lecture Notes in Computer Science, vol. 14083, pp. 729–761. Springer, Cham (Aug 2023). https://doi.org/10.1007/978-3-031-38548-3_24
44. Katsumata, S., Lai, Y.F., Reichle, M.: Breaking parallel ROS: Implication for isogeny and lattice-based blind signatures. In: Tang, Q., Teague, V. (eds.) PKC 2024: 27th International Conference on Theory and Practice of Public Key Cryptography, Part I. Lecture Notes in Computer Science, vol. 14601, pp. 319–351. Springer, Cham (Apr 2024). https://doi.org/10.1007/978-3-031-57718-5_11
45. Lai, Y.F.: Advanced Isogeny-based Cryptosystems. Ph.D. thesis, the University of Auckland (2023)
46. Lai, Y.F.: Cappybara and tsubaki: Verifiable random functions from group actions and isogenies. IACR Communications in Cryptology (CiC) 1(3), 1 (2024). <https://doi.org/10.62056/avr-11zn4>
47. Lai, Y.F., Galbraith, S.D., Delpech de Saint Guilhem, C.: Compact, efficient and UC-secure isogeny-based oblivious transfer. In: Canteaut, A., Standaert, F.X. (eds.) Advances in Cryptology – EUROCRYPT 2021, Part I. Lecture Notes

- in Computer Science, vol. 12696, pp. 213–241. Springer, Cham (Oct 2021). https://doi.org/10.1007/978-3-030-77870-5_8
48. Lai, Y.F., Meers, J., Nowakowski, J.: Hidden number problems in fiat-shamir based post-quantum signatures. Cryptology ePrint Archive, Paper 2025/913 (2025), <https://eprint.iacr.org/2025/913>
 49. Lyubashevsky, V., Nguyen, N.K., Plançon, M.: Lattice-based zero-knowledge proofs and applications: Shorter, simpler, and more general. In: Dodis, Y., Shrimpton, T. (eds.) Advances in Cryptology – CRYPTO 2022, Part II. Lecture Notes in Computer Science, vol. 13508, pp. 71–101. Springer, Cham (Aug 2022). https://doi.org/10.1007/978-3-031-15979-4_3
 50. Microsoft.: U-prove. <https://www.microsoft.com/en-us/research/project/u-prove/> (2024), accessed: 2025-08-05
 51. del Pino, R., Katsumata, S.: A new framework for more efficient round-optimal lattice-based (partially) blind signature via trapdoor sampling. In: Dodis, Y., Shrimpton, T. (eds.) Advances in Cryptology – CRYPTO 2022, Part II. Lecture Notes in Computer Science, vol. 13508, pp. 306–336. Springer, Cham (Aug 2022). https://doi.org/10.1007/978-3-031-15979-4_11
 52. Pointcheval, D., Stern, J.: Security arguments for digital signatures and blind signatures. Journal of Cryptology **13**(3), 361–396 (Jun 2000). <https://doi.org/10.1007/s001450010003>
 53. Tessaro, S., Zhu, C.: Short pairing-free blind signatures with exponential security. In: Dunkelman, O., Dziembowski, S. (eds.) Advances in Cryptology – EURO-CRYPT 2022, Part II. Lecture Notes in Computer Science, vol. 13276, pp. 782–811. Springer, Cham (May / Jun 2022). https://doi.org/10.1007/978-3-031-07085-3_27

A Cryptographic Group Action and Tanuki Frameworks

A.1 Cryptographic Group Actions

The following is almost a verbatim of [36].

The study of post-quantum group actions has seen growing interest in recent years. This line of research began in the commutative setting, notably through isogeny-based constructions [25], and has since demonstrated broad applicability to advanced cryptographic primitives [14,47,32,13,46]. The developments [7,29,19,8,23,10,9,24] have emerged in the non-commutative setting, with increasing attention in both code-based and lattice-based cryptography.

Definition 8 (Group action). *Let $(G, \cdot)$ be a group. We say G acts on a set X by a function $\star : G \times X \rightarrow X$ or $(G, X, \star)$ is a group action if it satisfies the following properties. 1) Identity: for any $x \in X$, we have $e \star x = x$ where e is the identity element of G . 2) Compatibility: for any $g, h \in G$ and any $x \in X$, we have $(gh) \star x = g \star (h \star x)$.*

Definition 9 (Cryptographic Group Action). *Let $(x, G, X, \star)$ be given where $(G, X, \star)$ defines a group action and $x \in X$. We say that $(x, G, X, \star)$ is a cryptographic group action if there exist computationally efficient algorithms for the following tasks:*

- **Unique set element representation:** *Given any element $x \in X$, compute a unique representation of x .*
- **Unique group element representation:** *Given any element $g \in G$, compute a unique representation of g .*
- **Group sampling:** *Sampling over G uniformly at random (or statistically).*
- **Group element inversion:** *Given any $g \in G$, compute its inverse g^{-1} .*
- **Group action:** *Given any $x \in X$ and $g \in G$, compute $g \star x$.*

A.2 Hardness Assumptions and the Algebraic Group Action Model

We introduce the assumptions we considered in this work.

Definition 10 (Group action inverse problem (GAIP/GA-DLog)). *Let $\Pi = (x, G, X, \star)$ be a cryptographic group action. The group action inverse problem asks, given $y = g \star x$ where $g \leftarrow G$ to find an element $g' \in G$ such that $g' \star x = y$. We define the advantage of a p.p.t. adversary $\mathcal{A}$ in solving GAIP with respect to Π as $\text{Adv}_{\Pi, \mathcal{A}}^{\text{GAIP}}(\lambda) := \Pr[\mathcal{A}(y) \in G \text{ such that } \mathcal{A}(y) \star x = y]$, where the probability is over the randomness of sampling over G and the internal randomness of $\mathcal{A}$.*

Definition 11 (One-More-Vectorization Problem (OMVP)). *Let $\Pi = (x, G, X, \star)$ be a cryptographic group action. We define the one-more vectorization problem game OMVP with respect to a p.p.t. adversary $\mathcal{A}$ in figure 4. We define the advantage of $\mathcal{A}$ in OMVP with respect to Π as $\text{Adv}_{\Pi, \mathcal{A}}^{\text{OMVP}}(\lambda) := \Pr[\text{OMVP}^{\mathcal{A}}(\lambda) = 1]$, where the probability is over the randomness of sampling over G in the game and the internal randomness of $\mathcal{A}$.*

$\text{OMVP}_H^A(\lambda)$	$\text{dLog}(y, z)$	$\text{Chall}()$
$C = []; k = 0;$	$k = k + 1;$	$g \leftarrow \$ G; y \leftarrow g \star x;$
$(x, X, G, \star) \leftarrow \Pi$	if $\exists g \text{ s.t. } g \star y = z$ then	$C = C \parallel [g]$
$C' \leftarrow \mathcal{A}^{\text{Chall, dLog}}(x, X, G, \star)$	return g	return y
return $(C \star x = C' \star x \wedge k < C)$	else return $\perp$	

Fig. 4. The game OMVP and the oracles Chall, dLog.

Though some novel attacks on variant assumptions of the linear equivalence problem have been reported in the literature [23,48], the assumption described above remains unaffected. In short, [23] shows an effective attack when a secret monomial map is applied twice on different codes, which gives sufficiently many linear equations to solve for the monomial map. [48] gives an attack when a bit of nonce is leaked, where the zero-knowledge comprises and the attacker can reduce key space effectively.

Definition 12 (Algebraic Algorithm). *Let the cryptographic group action $\Pi = (x, G, X, \star)$ be given. We say that an algorithm $\mathcal{A}$ is algebraic if, for any set element $y \in X$ that it outputs, it also outputs a group element g and an index $i \in \mathbb{N}$, i.e., $(y, (g, i)) \leftarrow \$ \mathcal{A}(\vec{x})$ such that $y = g_i \star x_i$.*

For an adversary $\mathcal{A}$ that has access to oracles during its runtime, we impose the above restriction on all group elements that it outputs to an oracle. Similarly, all group elements that $\mathcal{A}$ receives through oracle interactions are treated as inputs to $\mathcal{A}$; hence, such group elements become part of $\vec{x}$ when $\mathcal{A}$ outputs set elements (and hence algebraic coefficients) at a later point.

A.3 Tanuki 2

Let $(x, X, G, \star)$ be a cryptographic group action. Let α be a positive integer and $\kappa = \kappa(\lambda) \geq 2$, $\mu = \mu(\lambda) = \alpha \cdot \kappa(\lambda)$ be polynomial functions in the security parameter λ . Let $H : \{0, 1\}^* \rightarrow [\kappa]^\mu$ be a hash function with the following property: for any string m and $i, j \in [\kappa]$ the number of occurrences of i in $H(m)$ is equal to those of j .

KeyGen(λ): set $g_1 = id$ and sample group elements $g_2, \dots, g_\kappa \leftarrow \$ G^{\kappa-1}$. Output $sk = (g_2, \dots, g_\kappa)$ and $pk = (pk_1, \dots, pk_\kappa) = (x, g_2 \star x, \dots, g_\kappa \star x)$.

Signer₁(sk): sample group elements $h_1, \dots, h_\mu \leftarrow \$ G^\mu$. Compute $y_i = h_i \star x$ for each $i \in [\mu]$. Send $Y = (y_1, \dots, y_\mu)$ to User and keep $h_1, \dots, h_\mu$.

User₁(pk, Y, m): sample group elements $(r_1, \dots, r_\mu) \leftarrow \$ G^\mu$ and a random permutation $\pi : [\mu] \rightarrow [\mu]$. Compute $z_i = r_i \star y_i$ for all $i \in [\mu]$. Compute the challenge $C = H(z_{\pi(1)}, \dots, z_{\pi(\mu)}, m)$. Interpret C as a list $(c_1, \dots, c_\mu)$ and set $C' = (c_{\pi^{-1}(1)}, \dots, c_{\pi^{-1}(\mu)})$. Send C' to Signer.

Signer₂($sk, C', (h_1, \dots, h_\mu)$): Parse C' as $(c'_1, \dots, c'_\mu)$. For each $i \in [\mu]$ set $s'_i = h_i \cdot g_{c'_i}^{-1}$. Send $S' = (s'_1, \dots, s'_\mu)$ to the User.

User₂($pk, S', \pi, (r_1, z_1, \dots, r_\mu, z_\mu), m$): For each $i \in [\mu]$ set $s_i = r_i \cdot s'_i$. Output signature $\text{sig} = ((z_{\pi(1)}, \dots, z_{\pi(\mu)}), (s_{\pi(1)}, \dots, s_{\pi(\mu)}))$. Abort if it is invalid.

Verify(pk, m, sig): Parse $\text{sig} = ((z_1, \dots, z_\mu), (s_1, \dots, s_\mu))$ and compute the challenge $(c_1, \dots, c_\mu) = H(z_1, \dots, z_\mu, m)$. Output 1 if for all $i \in [\mu]$ the following holds: $s_i \star pk_{c_i} = z_i$. Otherwise, output 0.

Framework 1: Multi-key Blind Signature

A.4 Tanuki 3

Let **KeyGen** be the key generation function of the previous scheme. We introduce the algorithm **KeyRand**: on input a public key $pk = (pk_1, \dots, pk_\kappa)$ sample a vector $(t_1, \dots, t_\kappa) \leftarrow \$ G^\kappa$ and permutation $\sigma \leftarrow \$ S_\kappa$, randomize pk by computing $pk' = (t_{\sigma(1)} \star pk_{\sigma(1)}, \dots, t_{\sigma(\kappa)} \star pk_{\sigma(\kappa)})$ and output $(pk', (t_1, \dots, t_\kappa), \sigma)$.

Let $(x, X, G, \star)$ be a cryptographic group action. Let α be a positive integer and $\kappa = \kappa(\lambda) \geq 2$, $\mu = \mu(\lambda) = \alpha \cdot \kappa(\lambda)$ be polynomial functions in the security parameter λ . Let $H : \{0, 1\}^* \rightarrow [\kappa]^\mu$ be a hash function with the following property: for any string m and $i, j \in [\kappa]$ the number of occurrences of i in $H(m)$ is equal to those of j .

KeyGen(λ): set $g_1 = id$ and sample group elements $g_2, \dots, g_\kappa \leftarrow G^{\kappa-1}$. Output $sk = (g_2, \dots, g_\kappa)$ and $pk = (pk_1, \dots, pk_\kappa) = (x, g_2 \star x, \dots, g_\kappa \star x)$.

Signer₁(sk, pk): compute $pk', t', \sigma' \leftarrow \text{KeyRand}(pk)$. For $i \in [\kappa]$ set $g'_i = t'_{\sigma'(i)} \cdot g_{\sigma'(i)}$. Sample group elements $h_1, \dots, h_\mu \leftarrow G$ and compute $y_i = h_i \star x$ for each $i \in [\mu]$. Send $pk', (y_1, \dots, y_\mu)$ to User and keep $h_1, \dots, h_\mu, g'_1, \dots, g'_\kappa, t', \sigma'$.

User₁($pk', y_1, \dots, y_\mu, m$): compute $pk'', t, \sigma \leftarrow \text{KeyRand}(pk')$. Sample group elements $r_1, \dots, r_\mu \leftarrow G$ and random permutation π . For $i \in [\mu]$ compute $z_i = r_i \star y_i$. Compute the challenge $c = H(z_{\pi(1)}, \dots, z_{\pi(\mu)}, pk'', m)$. Interpret c as a list $c_1, \dots, c_\mu$ and set $c' = (\sigma(c_{\pi^{-1}(1)}), \dots, \sigma(c_{\pi^{-1}(\mu)}))$. Send c' to Signer.

Signer₂($c', g'_1, \dots, g'_\kappa, h_1, \dots, h_\mu, t', \sigma'$): parse c' as $(c'_1, \dots, c'_\mu)$. For each $i \in [\mu]$ set $s'_i = h_i \cdot (g'_{c'_i})^{-1}$. Send $S' = (s'_1, \dots, s'_\mu), t', \sigma'$ to the User.

User₂($pk, S', \pi, (r_1, z_1, \dots, r_\mu, z_\mu), m$): Verify $pk' \stackrel{?}{=} \text{KeyGen}(pk; t', \sigma')$ and verify S' . For each $i \in [\mu]$, compute $s_i = r_i \cdot s'_i \cdot t_{c'_i}^{-1}$. Output signature

$$\text{sig} = (c, (s_{\pi(1)}, \dots, s_{\pi(\mu)}), (t_{\sigma(1)} \cdot t'_{\sigma'(\sigma(1))}, \dots, t_{\sigma(\kappa)} \cdot t'_{\sigma'(\sigma(\kappa))}), \sigma' \circ \sigma).$$

Verify(pk, m, sig): Parse $\text{sig} = ((c_1, \dots, c_\mu), (s_1, \dots, s_\mu), (t_1, \dots, t_\kappa), \sigma)$. Compute $\overline{pk} = (t_1 \star pk_{\sigma(1)}, \dots, t_\kappa \star pk_{\sigma(\kappa)})$, and verify that $c \stackrel{?}{=} H(s_1 \star \overline{pk}_{c_1}, \dots, s_\mu \star \overline{pk}_{c_\mu}, \overline{pk}, m)$ holds. Output 1 if the above equality holds, and 0 otherwise.

Framework 2: Randomized Multi-key Blind Signature